

РЕФЕРАТ

Отчёт содержит 68 с., 16 рис., 16 табл., 9 источников.

ПЛАНИРОВАНИЕ СЕМЕЙНОГО БЮДЖЕТА, ВЕБ-ПРИЛОЖЕНИЕ, DJANGO, PYTHON, SQLITE3, DJANGO REST FRAMEWORK, MVT, REST API, SWAGGER UI, АВТОРИЗАЦИЯ, РОЛЕВАЯ МОДЕЛЬ, ФИНАНСОВЫЕ ТРАНЗАКЦИИ, БЮДЖЕТНЫЕ ОГРАНИЧЕНИЯ, АНАЛИТИКА РАСХОДОВ

Объект разработки — веб-приложение для планирования семейного бюджета.

Цель работы — разработка серверной части веб-приложения для планирования семейного бюджета с использованием языка программирования Python и фреймворка Django, обеспечивающего регистрацию и авторизацию пользователей, разграничение прав доступа, управление семейными группами, учёт доходов и расходов, контроль бюджетных ограничений, хранение финансовых данных и формирование отчётности.

Методы проведения работы — анализ предметной области управления личными и семейными финансами, сравнение существующих решений, проектирование архитектуры веб-приложения на основе паттерна MVT, разработка серверной логики на Django, реализация базы данных средствами SQLite3 и Django ORM, создание REST API с использованием Django REST Framework, документирование API через OpenAPI и Swagger UI, а также проведение модульного, интеграционного и ручного тестирования.

Результаты работы — разработано веб-приложение для планирования семейного бюджета, реализованы регистрация и авторизация пользователей, управление семейными группами, разграничение прав доступа, работа с категориями доходов и расходов, добавление и редактирование финансовых транзакций, настройка бюджетных ограничений, импорт и экспорт данных в формате CSV, формирование финансовой отчётности, административная панель Django, REST API и документация API через Swagger UI.

Оглавление

РЕФЕРАТ	1
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	5
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
Введение	7
1 Анализ предметной области	9
1.1 Общая характеристика предметной области	9
1.2 Анализ существующих решений	10
1.3 Сравнительный анализ аналогов	10
1.4 Описание пользователей системы	11
1.5 Диаграмма вариантов использования системы	12
1.6 Функциональные требования к системе	14
1.7 Нефункциональные требования к системе	15
1.8 Контекстная диаграмма IDEF0	16
1.9 Основные сущности системы	17
1.10 Вывод по разделу	18
2 Обоснование выбора технологий разработки веб-приложения	20
2.1 Общая характеристика используемых технологий	20
2.2 Выбор языка программирования Python	20
2.3 Выбор фреймворка Django	21
2.4 Использование архитектурного шаблона MVT	22
2.5 Выбор системы управления базами данных SQLite	23
2.6 Использование Django Templates	23
2.7 Использование Django REST Framework и Swagger/OpenAPI	24
2.8 Выбор среды разработки Visual Studio Code и системы контроля версий Git	25

2.9 Вывод по разделу	25
3 Проектирование архитектуры веб-приложения	27
3.1 Общая архитектура веб-приложения	27
3.2 Использование паттерна MVT.....	28
3.3 Структура проекта.....	29
3.4 Организация маршрутизации	30
3.5 Архитектура системы авторизации и разграничения прав доступа	31
3.6 Архитектура REST API	32
3.7 Взаимодействие компонентов системы	33
3.8 Вывод по разделу	34
4 Разработка серверной части веб-приложения	35
4.1 Общая характеристика серверной части.....	35
4.2 Реализация пользовательской модели и авторизации	36
4.3 Реализация управления семейными группами	37
4.4 Реализация управления категориями финансовых операций	37
4.5 Реализация управления финансовыми транзакциями	38
4.6 Реализация бюджетных ограничений	39
4.7 Реализация отчетности и аналитики	40
4.8 Реализация импорта и экспорта данных	41
4.9 Реализация комментариев и истории изменений	41
4.10 Реализация REST API и документации Swagger/OpenAPI.....	42
4.11 Вывод по разделу	43
5 Реализация базы данных	44
5.1 Общая характеристика базы данных.....	44
5.2 Графическое представление базы данных	44

5.3 Основные сущности базы данных	46
5.4 Описание сущностей пользователей и семейных групп	46
5.5 Описание сущностей финансового учета	47
5.6 Описание сущностей контроля и сопровождения данных.....	48
5.7 Связи между сущностями	49
5.8 Использование Django ORM и миграций.....	49
5.9 Обеспечение целостности данных	50
5.10 Вывод по разделу	51
6 Разработка клиентского слоя веб-приложения.....	52
6.1 Общая характеристика клиентского слоя	52
6.2 Структура пользовательского интерфейса	52
6.3 Реализация шаблонов Django	53
6.4 Навигация веб-приложения	53
6.5 Вывод по разделу	54
7 Тестирование веб-приложения	55
7.1 Цели и организация тестирования.....	55
7.2 Модульное и интеграционное тестирование	55
7.3 Тестирование веб-интерфейса	57
7.4 Тестирование API через Swagger UI	60
7.5 Тестирование API с использованием Postman	62
7.6 Результаты тестирования	65
Заключение	67
Список используемой литературы	68

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В отчёте по курсовой работе применяются следующие термины с соответствующими определениями:

Веб-приложение — программная система, доступ к которой осуществляется через браузер с использованием сетевых протоколов.

Серверная часть — часть приложения, отвечающая за обработку запросов, выполнение бизнес-логики, взаимодействие с базой данных и формирование ответов пользователю.

Клиентский слой — часть веб-приложения, обеспечивающая отображение интерфейса и взаимодействие пользователя с системой.

REST API — программный интерфейс приложения, основанный на HTTP-запросах и предназначенный для обмена данными между клиентом и сервером.

Swagger UI — инструмент для визуального просмотра и тестирования REST API на основе OpenAPI-схемы.

ORM — технология, позволяющая работать с таблицами базы данных через объекты языка программирования.

Миграция базы данных — механизм управления изменениями структуры базы данных при изменении моделей приложения.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчёте по курсовой работе применяют следующие сокращения и обозначения:

API — Application Programming Interface, программный интерфейс приложения.

CRUD — Create, Read, Update, Delete, создание, чтение, обновление и удаление данных.

CSV — Comma-Separated Values, текстовый формат представления табличных данных.

CSS — Cascading Style Sheets, каскадные таблицы стилей.

DRF — Django REST Framework, библиотека для разработки REST API во фреймворке Django.

HTML — HyperText Markup Language, язык гипертекстовой разметки.

HTTP — HyperText Transfer Protocol, протокол передачи гипертекста.

IDEF0 — методология функционального моделирования, используемая для описания процессов системы.

JSON — JavaScript Object Notation, текстовый формат обмена данными.

MVT — Model–View–Template, архитектурный паттерн Django, разделяющий приложение на модель, представление и шаблон.

ORM — Object-Relational Mapping, объектно-реляционное отображение.

REST — Representational State Transfer, архитектурный стиль взаимодействия компонентов распределённой системы.

SQL — Structured Query Language, язык структурированных запросов.

SQLite3 — встроенная реляционная система управления базами данных, используемая в проекте.

Введение

В условиях активного развития цифровых технологий и широкого распространения веб-приложений возрастает необходимость автоматизации процессов управления личными и семейными финансами. Контроль доходов и расходов позволяет пользователям более эффективно распределять денежные средства, анализировать структуру затрат и планировать долгосрочные финансовые цели. Использование специализированных программных решений значительно упрощает процесс ведения бюджета и снижает вероятность возникновения финансовых ошибок.

Существующие системы учета финансов зачастую ориентированы преимущественно на индивидуальное использование и не обеспечивают полноценной поддержки совместного управления семейным бюджетом. Кроме того, многие решения обладают ограниченными возможностями разграничения прав доступа между участниками семьи, а также не предоставляют удобных инструментов анализа финансовых операций и контроля бюджетных ограничений.

Актуальность разработки серверной части веб-приложения для планирования семейного бюджета обусловлена необходимостью создания централизованной системы хранения и обработки финансовых данных, обеспечивающей совместную работу нескольких пользователей в рамках единого бюджета. Использование веб-технологий позволяет обеспечить доступ к системе с различных устройств через интернет, а серверная архитектура обеспечивает централизованную обработку данных, управление доступом пользователей и хранение информации в базе данных.

Целью курсовой работы является разработка серверной части веб-приложения для планирования семейного бюджета с использованием языка программирования Python и фреймворка Django.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих аналогов систем

управления личными финансами;

- обосновать выбор технологий разработки веб-приложения;
- разработать архитектуру программной системы;
- реализовать серверную логику обработки пользовательских запросов;
- разработать структуру базы данных и реализовать взаимодействие с ней;
- реализовать клиентский слой веб-приложения;
- выполнить тестирование разработанной системы.

Объектом исследования является процесс управления семейным бюджетом.

Предметом исследования являются методы и средства разработки серверной части веб-приложений для автоматизации учета финансовых операций.

Практическая значимость работы заключается в создании программной системы, обеспечивающей централизованное хранение финансовых данных, учет доходов и расходов, контроль бюджетных ограничений и совместное использование семейного бюджета несколькими пользователями.

1 Анализ предметной области

1.1 Общая характеристика предметной области

Управление семейным бюджетом представляет собой процесс учета доходов и расходов нескольких пользователей в рамках единой финансовой системы. Основной задачей подобных систем является централизованное хранение информации о финансовых операциях, контроль расходов, анализ структуры затрат и планирование бюджета.

В настоящее время для ведения личных финансов широко используются мобильные и веб-приложения, позволяющие автоматизировать учет денежных средств. Однако большинство существующих решений ориентировано преимущественно на индивидуальное использование и не обеспечивает полноценной поддержки совместного управления семейным бюджетом.

При совместном использовании финансовой системы возникает необходимость разграничения прав доступа между участниками семьи, объединения финансовых операций в рамках общей группы, контроля лимитов расходов и формирования общей финансовой статистики. Кроме того, пользователям требуется возможность централизованного хранения данных, просмотра истории операций и получения отчетов о состоянии бюджета.

Разрабатываемое веб-приложение предназначено для автоматизации процессов учета семейных финансов. Система обеспечивает регистрацию пользователей, создание семейных групп, учет доходов и расходов, управление категориями финансовых операций, установку бюджетных ограничений и формирование отчетности.

Серверная часть приложения реализует обработку HTTP-запросов, взаимодействие с базой данных, проверку прав доступа пользователей и выполнение бизнес-логики системы. Для хранения данных используется реляционная база данных SQLite, а взаимодействие с клиентской частью реализовано средствами фреймворка Django.

1.2 Анализ существующих решений

Перед разработкой системы был проведен анализ существующих приложений для учета личных и семейных финансов.

В качестве аналогов были рассмотрены следующие системы: CoinKeeper, Zenmoney, Monefy.

Система CoinKeeper предоставляет возможность учета доходов и расходов, распределения операций по категориям и анализа финансовой статистики. Приложение обладает удобным интерфейсом и поддерживает синхронизацию данных между устройствами. Недостатком является ограниченная поддержка совместного использования семейного бюджета и отсутствие гибкого разграничения ролей пользователей.

Система Zenmoney ориентирована на ведение личных финансов и обладает широкими возможностями анализа расходов. Поддерживается синхронизация с банковскими сервисами и автоматическая обработка транзакций. Недостатком является зависимость части функциональности от внешних сервисов и ограниченные возможности управления ролями участников семейной группы.

Приложение Monefy предназначено для быстрого учета доходов и расходов и отличается простотой использования. Основной недостаток системы заключается в отсутствии развитых средств совместной работы пользователей и ограниченных возможностях аналитики.

1.3 Сравнительный анализ аналогов

Результаты анализа существующих решений представлены в таблице 1.

Таблица 1 - Сравнение существующих систем учета финансов

Функциональная возможность	CoinKeeper	Zenmoney	Monefy	Разрабатываемая система
Учет доходов и расходов	Да	Да	Да	Да
Совместное использование семейного бюджета	Частично	Частично	Нет	Да
Разграничение ролей пользователей	Нет	Нет	Нет	Да
Веб-интерфейс	Да	Да	Нет	Да
Управление категориями операций	Да	Да	Да	Да
Установка лимитов бюджета	Да	Да	Нет	Да

Продолжение таблицы 1

Импорт и экспорт CSV	Ограниченно	Да	Нет	Да
Серверная обработка данных	Да	Да	Нет	Да
Открытая архитектура для расширения	Нет	Нет	Нет	Да

На основании проведенного анализа можно сделать вывод, что существующие решения в основном ориентированы на индивидуальный учет финансов и не обеспечивают полноценной поддержки совместного управления семейным бюджетом. Основными недостатками рассмотренных систем являются ограниченные возможности разграничения прав доступа, отсутствие гибкой серверной архитектуры и недостаточная поддержка совместной работы пользователей.

Разрабатываемое веб-приложение учитывает выявленные недостатки и предоставляет централизованную серверную систему управления семейными финансами с поддержкой ролей пользователей, бюджетных ограничений, формирования отчетов и совместного использования данных несколькими участниками семьи.

1.4 Описание пользователей системы

В разрабатываемом веб-приложении предусмотрено несколько категорий пользователей, взаимодействующих с системой управления семейным бюджетом. Разделение пользователей по ролям необходимо для ограничения доступа к финансовым данным, защиты информации и организации совместной работы в рамках одной семейной группы.

Участник семейной группы получает доступ к общему финансовому пространству семьи. Он может добавлять доходы и расходы, просматривать доступные категории, участвовать в формировании общей статистики и отслеживать выполнение бюджетных ограничений. При этом права участника могут быть ограничены по сравнению с владельцем группы, что позволяет обеспечить разграничение доступа.

Отдельную роль в системе занимает владелец семейной группы. Данный

пользователь создает семейную группу, приглашает или добавляет участников, управляет общими категориями, контролирует бюджетные ограничения и имеет доступ к общей финансовой статистике семьи. Владелец семейной группы отвечает за организацию совместного учета финансов и настройку основных параметров группы.

Также в системе предусмотрен администратор. Администратор осуществляет техническое сопровождение приложения через административную панель Django, управляет учетными записями пользователей, просматривает данные системы и выполняет служебные операции, необходимые для корректной работы веб-приложения.

Таким образом, система ролей позволяет организовать совместную работу пользователей, обеспечить контроль доступа к финансовым данным и повысить безопасность хранения информации.

1.5 Диаграмма вариантов использования системы

Для наглядного представления взаимодействия пользователей с разрабатываемым веб-приложением была построена диаграмма вариантов использования. Данная диаграмма позволяет отразить основные роли пользователей системы и функции, доступные каждой категории пользователей.

В разрабатываемой системе основными участниками являются владелец семейной группы, участник семейной группы и администратор. Каждый из них выполняет определенные действия в рамках системы планирования семейного бюджета.

Владелец семейной группы обладает расширенными возможностями. Он может создавать семейную группу, добавлять участников, управлять общими категориями, устанавливать бюджетные ограничения и просматривать общую финансовую статистику семьи. Данная роль необходима для организации совместного ведения семейного бюджета.

Участник семейной группы может добавлять доходы и расходы, просматривать доступные категории, участвовать в формировании общей

статистики и контролировать выполнение бюджетных ограничений. При этом его права могут быть ограничены по сравнению с владельцем семейной группы.

Администратор взаимодействует с системой через административную панель Django. Он может управлять учетными записями пользователей, просматривать данные системы, контролировать корректность работы приложения и выполнять служебные операции.



Рисунок 1 – Use Case диаграмма взаимодействия пользователей с системой

Представленная диаграмма вариантов использования показывает, какие функции доступны каждой роли пользователя. Она позволяет уточнить границы системы, определить основные сценарии взаимодействия и связать пользователей приложения с функциональными требованиями.

1.6 Функциональные требования к системе

Функциональные требования определяют основные возможности, которые должно предоставлять разрабатываемое веб-приложение для планирования семейного бюджета. Система должна обеспечивать регистрацию пользователей, управление финансовыми операциями, работу с семейными группами, формирование аналитики и взаимодействие с серверной частью через веб-интерфейс и API.

Таблица 2 – Функциональные требования к системе

№	Функциональное требование	Описание
1	Регистрация пользователей	Система должна предоставлять возможность создания новой учетной записи пользователя.
2	Авторизация пользователей	Система должна обеспечивать вход пользователя в личный кабинет с использованием учетных данных.
3	Управление семейными группами	Пользователь должен иметь возможность создать семейную группу и участвовать в совместном ведении бюджета.
4	Разграничение прав доступа	Система должна различать владельца семейной группы, участника группы и администратора.
5	Управление категориями операций	Пользователь должен иметь возможность создавать, изменять и удалять категории доходов и расходов.
6	Учет финансовых транзакций	Система должна позволять добавлять, редактировать и удалять финансовые операции.
7	Разделение операций по типам	Финансовые операции должны подразделяться на доходы и расходы.
8	Управление бюджетными ограничениями	Пользователь должен иметь возможность устанавливать лимиты расходов по категориям за определенный период.
9	Формирование отчетности	Система должна предоставлять сводную информацию о доходах, расходах и состоянии бюджета.
10	Анализ финансовых данных	Система должна обеспечивать отображение статистики по категориям, периодам и пользователям.
11	Импорт и экспорт данных	Система должна поддерживать импорт и экспорт финансовых данных в формате CSV.
12	Комментарии к записям	Пользователь должен иметь возможность оставлять комментарии к финансовым операциям или связанным объектам системы.
13	История изменений	Система должна фиксировать изменения важных данных, таких как сумма, категория, дата операции или бюджетное ограничение.
14	Работа с REST API	Серверная часть должна предоставлять API для взаимодействия с данными приложения.
15	Документирование API	Система должна поддерживать автоматическую документацию API с использованием Swagger/OpenAPI.
16	Административная панель	Администратор должен иметь доступ к управлению данными через встроенную административную панель.

Реализация перечисленных требований позволяет обеспечить полный цикл работы с семейным бюджетом: от регистрации пользователя до анализа финансовой статистики и контроля расходов. Наличие REST API и документации Swagger упрощает тестирование серверной части и создает основу для дальнейшего расширения системы.

1.7 Нефункциональные требования к системе

Нефункциональные требования определяют качественные характеристики разрабатываемого веб-приложения. Они не описывают отдельные функции системы, но задают требования к надежности, безопасности, удобству использования и дальнейшему сопровождению программного продукта.

Таблица 3 – Нефункциональные требования к системе

№	Нефункциональное требование	Описание
1	Безопасность	Система должна обеспечивать защиту пользовательских данных, использовать механизмы авторизации и ограничивать доступ к чужим финансовым данным.
2	Целостность данных	Информация о пользователях, семейных группах, категориях, транзакциях и бюджетах должна храниться согласованно за счет связей между таблицами базы данных.
3	Производительность	Система должна обеспечивать выполнение основных пользовательских операций без заметных задержек при учебной нагрузке.
4	Удобство использования	Интерфейс приложения должен быть понятным для пользователя и обеспечивать быстрый доступ к основным функциям учета бюджета.
5	Поддерживаемость	Код приложения должен иметь модульную структуру, что упрощает внесение изменений и добавление новых функций.
6	Масштабируемость	Архитектура системы должна позволять в дальнейшем расширять функциональность, например добавлять новые виды отчетов или интеграции.
7	Совместимость	Веб-приложение должно корректно работать в современных браузерах и не требовать установки дополнительного программного обеспечения на стороне пользователя.
8	Документируемость	Серверное API должно быть описано с помощью Swagger/OpenAPI для упрощения тестирования и сопровождения.
9	Тестируемость	Основные функции серверной части должны быть доступны для проверки с помощью автоматических тестов и API-инструментов.

Соблюдение нефункциональных требований позволяет повысить качество разрабатываемого программного продукта. Особое значение для системы управления семейным бюджетом имеют безопасность, целостность данных, так как приложение работает с финансовой информацией пользователей.

1.8 Контекстная диаграмма IDEF0

Для наглядного представления функций разрабатываемого веб-приложения была построена контекстная диаграмма IDEF0. Данная диаграмма позволяет представить систему планирования семейного бюджета как единый процесс, связанный с обработкой пользовательских данных, финансовых операций, категорий расходов и бюджетных ограничений.

На контекстной диаграмме основной функцией системы является «Управление семейным бюджетом».

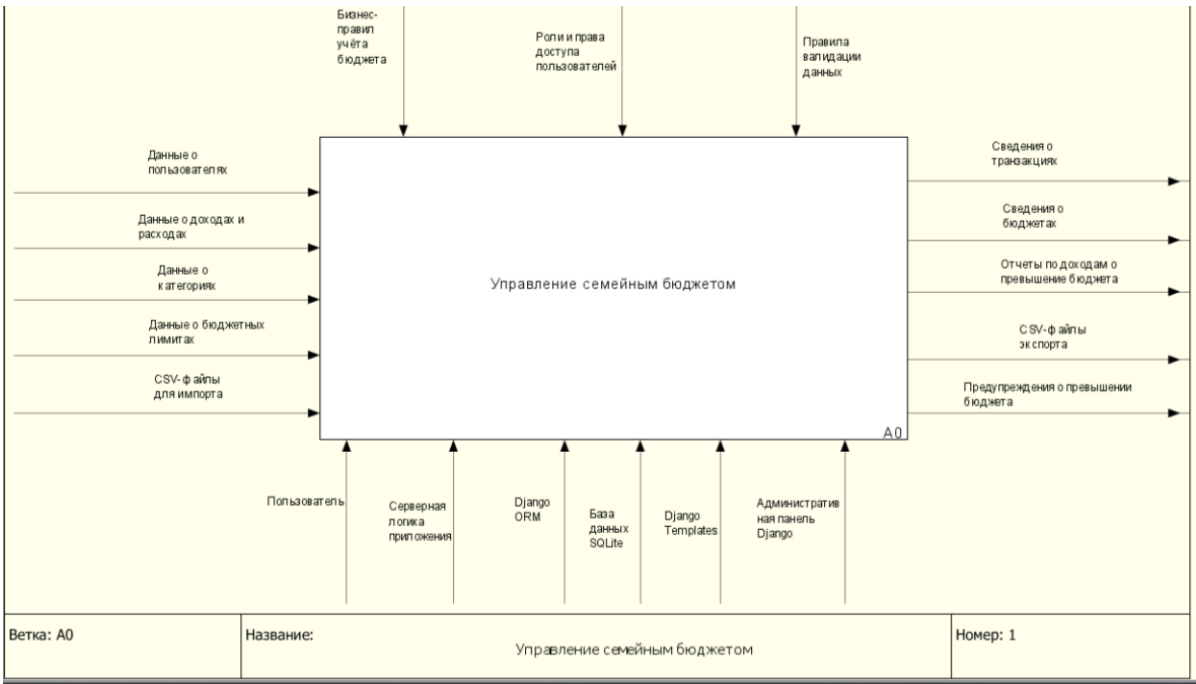


Рисунок 2 – Контекстная диаграмма IDEF0 системы планирования семейного бюджета

Контекстная диаграмма отражает общую логику работы системы и показывает, какие данные поступают на вход, какие правила влияют на выполнение процесса, какие средства используются для его реализации и какие результаты получает пользователь после обработки информации.

Для более детального описания работы системы была выполнена декомпозиция основной функции. Декомпозиция позволяет разделить общий процесс управления семейным бюджетом на несколько связанных подпроцессов, каждый из которых отвечает за отдельную часть функциональности веб-приложения.

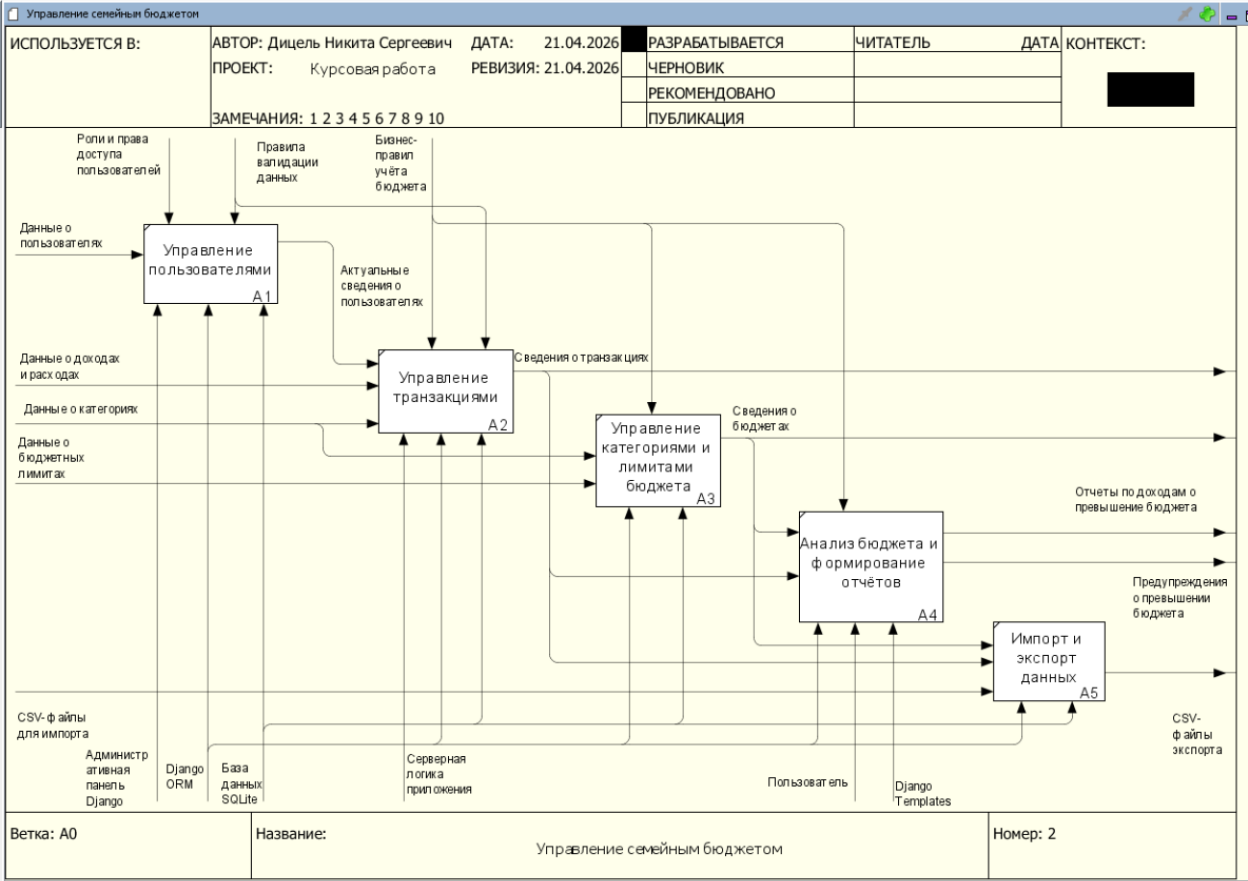


Рисунок 3 – Декомпозиция контекстной диаграммы IDEF0

1.9 Основные сущности системы

В разрабатываемом веб-приложении для планирования семейного бюджета используются несколько основных сущностей, которые отражают ключевые объекты предметной области. Данные сущности необходимы для хранения информации о пользователях, семейных группах, финансовых операциях, категориях и бюджетных ограничениях.

Таблица 4 – Основные сущности системы

Сущность	Назначение
Пользователь	Хранение учетных данных и идентификация пользователя в системе
Семейная группа	Объединение нескольких пользователей для совместного ведения бюджета

Продолжение таблицы 4

Участник семьи	Связь пользователя с семейной группой и хранение его роли
Категория	Группировка доходов и расходов по типам финансовых операций
Транзакция	Хранение финансовой операции: дохода или расхода
Бюджет	Хранение лимита расходов по категории за определенный период
Комментарий	Добавление пояснений к объектам системы или финансовым операциям

Выделенные сущности формируют основу предметной области разрабатываемой системы. Они позволяют организовать учет семейных финансов, обеспечить совместную работу пользователей, структурировать финансовые операции и контролировать выполнение бюджетных ограничений.

Подробное описание структуры данных, связей между сущностями и особенностей их реализации рассматривается далее в разделе, посвященном проектированию и реализации базы данных.

1.10 Вывод по разделу

В результате анализа предметной области были рассмотрены особенности систем управления семейным бюджетом и выявлены основные требования к разрабатываемому веб-приложению. Проведенный анализ существующих решений показал, что большинство аналогов ориентировано преимущественно на индивидуальное использование и обладает ограниченными возможностями совместного управления финансами.

На основании сравнительного анализа была определена необходимость разработки системы, обеспечивающей централизованное хранение финансовых данных, разграничение прав доступа пользователей, управление семейными группами, контроль бюджетных ограничений и формирование финансовой отчетности.

В разделе были выделены основные категории пользователей системы, включая зарегистрированного пользователя, владельца семейной группы, участника семейной группы и администратора. Также были сформулированы функциональные и нефункциональные требования к веб-приложению,

определяющие как основные возможности системы, так и качественные характеристики программного продукта.

Кроме того, были определены основные сущности предметной области и взаимосвязи между ними, что позволило сформировать основу для дальнейшего проектирования архитектуры, структуры базы данных и серверной логики веб-приложения.

2 Обоснование выбора технологий разработки веб-приложения

2.1 Общая характеристика используемых технологий

Для разработки серверной части веб-приложения планирования семейного бюджета был выбран набор технологий, обеспечивающий быструю разработку, удобное сопровождение и возможность дальнейшего расширения системы. Основу проекта составляют язык программирования Python, веб-фреймворк Django, встроенная реляционная база данных SQLite, шаблонизатор Django Templates, а также средства разработки Visual Studio Code и система контроля версий Git.

Выбранный технологический стек соответствует задачам курсовой работы, так как позволяет реализовать регистрацию и авторизацию пользователей, управление семейными группами, учет финансовых транзакций, работу с категориями, бюджетными ограничениями, отчетностью и API. Кроме того, Django предоставляет встроенные механизмы безопасности, административную панель, ORM для работы с базой данных и удобную систему маршрутизации HTTP-запросов.

Таблица 5 – Используемые технологии разработки

Технология	Назначение в проекте
Python	Реализация серверной логики веб-приложения
Django	Основной фреймворк для разработки серверной части
Django Templates	Формирование HTML-страниц пользовательского интерфейса
Django ORM	Работа с базой данных через объектную модель
SQLite	Хранение данных приложения
Django REST Framework	Реализация серверного API
drf-spectacular	Формирование OpenAPI-схемы и Swagger-документации
Visual Studio Code	Среда разработки проекта
Git	Контроль версий исходного кода

Использование данных технологий позволяет создать веб-приложение, соответствующее требованиям учебного проекта и обладающее достаточной гибкостью для последующего развития.

2.2 Выбор языка программирования Python

В качестве языка программирования для разработки серверной части

веб-приложения был выбран Python. Данный язык широко применяется при создании веб-приложений благодаря простому синтаксису, высокой читаемости кода и большому количеству готовых библиотек.

Python поддерживает объектно-ориентированный подход к разработке программного обеспечения, что позволяет удобно описывать сущности предметной области, такие как пользователь, семейная группа, категория, транзакция и бюджетное ограничение. Также Python хорошо подходит для реализации бизнес-логики, обработки пользовательских запросов, выполнения проверок и взаимодействия с базой данных.

Одним из преимуществ Python является развитая экосистема веб-разработки. Для данного языка существует большое количество фреймворков и инструментов, позволяющих ускорить разработку программного продукта. В рамках курсовой работы это особенно важно, так как необходимо не только реализовать функциональность приложения, но и обеспечить его тестирование, документирование и сопровождение.

Использование Python позволяет сократить объем программного кода, повысить его читаемость и упростить дальнейшую модификацию системы.

2.3 Выбор фреймворка Django

Для разработки веб-приложения был выбран фреймворк Django. Django является одним из наиболее распространенных фреймворков для разработки серверных веб-приложений на языке Python. Он предоставляет готовые инструменты для маршрутизации запросов, работы с базой данных, обработки форм, авторизации пользователей и формирования HTML-страниц.

Одним из ключевых преимуществ Django является наличие встроенной ORM-системы. ORM позволяет описывать таблицы базы данных в виде Python-классов и выполнять операции с данными без ручного написания SQL-запросов. Это упрощает разработку, снижает вероятность ошибок и делает код более понятным.

Django также содержит встроенные механизмы безопасности. Фреймворк предоставляет защиту от распространенных уязвимостей веб-

приложений, включая CSRF-атаки, SQL-инъекции и небезопасную обработку пользовательских данных. Для системы управления семейным бюджетом это особенно важно, так как приложение работает с финансовой информацией пользователей.

Дополнительным преимуществом Django является встроенная административная панель. Она позволяет управлять данными приложения, просматривать пользователей, семейные группы, категории, транзакции и другие сущности без необходимости разрабатывать отдельный интерфейс администратора с нуля.

Таким образом, Django является подходящим инструментом для реализации серверной части веб-приложения, так как объединяет в себе средства разработки, безопасности, администрирования и работы с базой данных.

2.4 Использование архитектурного шаблона MVT

Фреймворк Django основан на архитектурном шаблоне MVT, который включает три основных компонента: Model, View и Template. Использование данного шаблона позволяет разделить логику приложения на отдельные части и упростить сопровождение программного кода.

Компонент Model отвечает за описание структуры данных и взаимодействие с базой данных. В рамках разрабатываемого приложения модели используются для хранения информации о пользователях, семейных группах, участниках семьи, категориях, транзакциях, бюджетах, комментариях и истории изменений.

Компонент View отвечает за обработку HTTP-запросов и реализацию бизнес-логики. Представления получают запросы от пользователя, выполняют необходимые проверки, обращаются к моделям и передают данные в шаблоны или API-ответы.

Компонент Template используется для формирования HTML-страниц, отображаемых пользователю в браузере. Шаблоны позволяют отделить внешний вид страниц от серверной логики приложения.

Применение шаблона MVT делает структуру проекта более понятной и логичной. Это упрощает добавление новых функций, изменение существующих модулей и тестирование отдельных частей системы.

2.5 Выбор системы управления базами данных SQLite

В качестве системы управления базами данных была выбрана SQLite. Данная СУБД является встроенной, не требует установки отдельного серверного программного обеспечения и хорошо интегрируется с Django.

SQLite хранит данные в отдельном файле базы данных, что упрощает настройку проекта и его запуск в учебной среде. Для курсовой работы такой вариант является оптимальным, так как приложение не рассчитано на высокую промышленную нагрузку и большое количество одновременных пользователей.

В базе данных веб-приложения хранятся сведения о пользователях, семейных группах, участниках семей, категориях финансовых операций, транзакциях, бюджетных ограничениях, комментариях и истории изменений. Связи между сущностями реализуются с помощью внешних ключей, что обеспечивает целостность и согласованность данных.

К преимуществам SQLite можно отнести простоту настройки, отсутствие необходимости администрирования отдельного сервера базы данных, высокую скорость работы при небольших нагрузках и полную поддержку со стороны Django ORM.

Таким образом, SQLite соответствует требованиям учебного веб-приложения и позволяет сосредоточиться на реализации бизнес-логики системы без усложнения процесса развертывания.

2.6 Использование Django Templates

Для реализации клиентского представления веб-приложения используются Django Templates. Данный шаблонизатор входит в состав Django и позволяет формировать HTML-страницы на стороне сервера.

Django Templates применяются для отображения данных, полученных из

базы данных, в удобном для пользователя виде. С их помощью реализуются страницы регистрации и авторизации, личный кабинет пользователя, списки финансовых операций, формы добавления транзакций, страницы управления категориями, бюджетами и отчетностью.

Использование серверных шаблонов является рациональным решением для курсового проекта, так как позволяет реализовать пользовательский интерфейс без необходимости разработки отдельного frontend-приложения. Это упрощает архитектуру системы и снижает сложность взаимодействия между клиентской и серверной частями.

Шаблоны Django также поддерживают наследование, что позволяет создавать общую структуру страниц и повторно использовать элементы интерфейса, например меню навигации, заголовки, формы и блоки вывода сообщений.

2.7 Использование Django REST Framework и Swagger/OpenAPI

Для реализации серверного API в проекте используется Django REST Framework. Данная библиотека расширяет возможности Django и позволяет создавать API для взаимодействия с данными приложения в формате JSON.

REST API используется для обработки запросов, связанных с финансовыми данными, категориями, бюджетами, комментариями и историей изменений. Наличие API делает серверную часть более гибкой и создает основу для дальнейшего расширения проекта, например подключения мобильного приложения или отдельного frontend-интерфейса.

Для документирования API используется библиотека drf-spectacular. Она позволяет автоматически формировать OpenAPI-схему и предоставляет интерфейс Swagger UI. Благодаря этому можно просматривать список доступных endpoint'ов, изучать структуру запросов и ответов, а также выполнять тестовые запросы прямо через браузер.

Использование Swagger/OpenAPI повышает удобство тестирования и сопровождения серверной части приложения. Документация API делает проект более понятным как для разработчика, так и для потенциальных

пользователей или проверяющих.

2.8 Выбор среды разработки Visual Studio Code и системы контроля версий Git

Разработка веб-приложения выполнялась в среде Visual Studio Code. Данная среда разработки поддерживает язык Python, работу с Django-проектами, подсветку синтаксиса, автодополнение кода, встроенный терминал и подключение расширений.

Visual Studio Code удобен для разработки учебных проектов, так как сочетает простоту интерфейса и широкий набор инструментов. Встроенный терминал позволяет выполнять команды Django, запускать сервер разработки, применять миграции базы данных и запускать тесты без перехода в отдельную командную строку.

Для контроля версий проекта использовалась система Git. Она позволяет отслеживать изменения исходного кода, возвращаться к предыдущим версиям проекта, фиксировать этапы разработки и организовывать резервное хранение программного продукта.

Использование Git повышает надежность процесса разработки, так как все изменения проекта могут быть сохранены в виде отдельных коммитов. Это особенно важно при постепенной реализации функциональности приложения и исправлении ошибок.

2.9 Вывод по разделу

В результате анализа технологий разработки был выбран программный стек, включающий Python, Django, SQLite, Django Templates, Django REST Framework, drf-spectacular, Visual Studio Code и Git.

Выбор Python и Django позволил реализовать серверную часть веб-приложения с использованием готовых механизмов маршрутизации, авторизации, работы с базой данных, шаблонов и административной панели. Применение архитектурного шаблона MVT обеспечило логическое разделение данных, бизнес-логики и пользовательского интерфейса.

Использование SQLite позволило упростить настройку базы данных и обеспечить надежное хранение информации в рамках учебного проекта. Django Templates были выбраны для формирования HTML-страниц, а Django REST Framework и Swagger/OpenAPI — для реализации и документирования серверного API.

Выбранные технологии соответствуют функциональным и нефункциональным требованиям разрабатываемого веб-приложения. Они обеспечивают удобство разработки, безопасность, поддерживаемость и возможность дальнейшего расширения системы планирования семейного бюджета.

3 Проектирование архитектуры веб-приложения

3.1 Общая архитектура веб-приложения

Разрабатываемое веб-приложение для планирования семейного бюджета построено на основе клиент-серверной архитектуры. Такой подход позволяет разделить пользовательский интерфейс, серверную бизнес-логику и хранение данных на отдельные части системы.

Клиентская часть представлена веб-интерфейсом, с которым пользователь взаимодействует через браузер. Через пользовательский интерфейс выполняются регистрация и авторизация, просмотр финансовых операций, добавление доходов и расходов, управление категориями, настройка бюджетных ограничений и просмотр отчетности.

Серверная часть реализована с использованием фреймворка Django. Она отвечает за обработку HTTP-запросов, проверку прав доступа, выполнение бизнес-логики, работу с базой данных и формирование HTML-страниц. Также серверная часть предоставляет REST API, предназначенный для взаимодействия с данными приложения и тестирования функциональности через Swagger/OpenAPI.

Для хранения данных используется реляционная база данных SQLite. В ней сохраняются сведения о пользователях, семейных группах, участниках семьи, категориях финансовых операций, транзакциях, бюджетах, комментариях и истории изменений. Связь между серверной частью и базой данных осуществляется с помощью ORM Django, что позволяет работать с таблицами базы данных через объектную модель.

Общая архитектура приложения включает три основных уровня: уровень представления, уровень бизнес-логики и уровень хранения данных. Уровень представления отвечает за отображение интерфейса пользователю. Уровень бизнес-логики выполняет обработку операций и проверку условий. Уровень хранения данных обеспечивает долговременное сохранение информации.

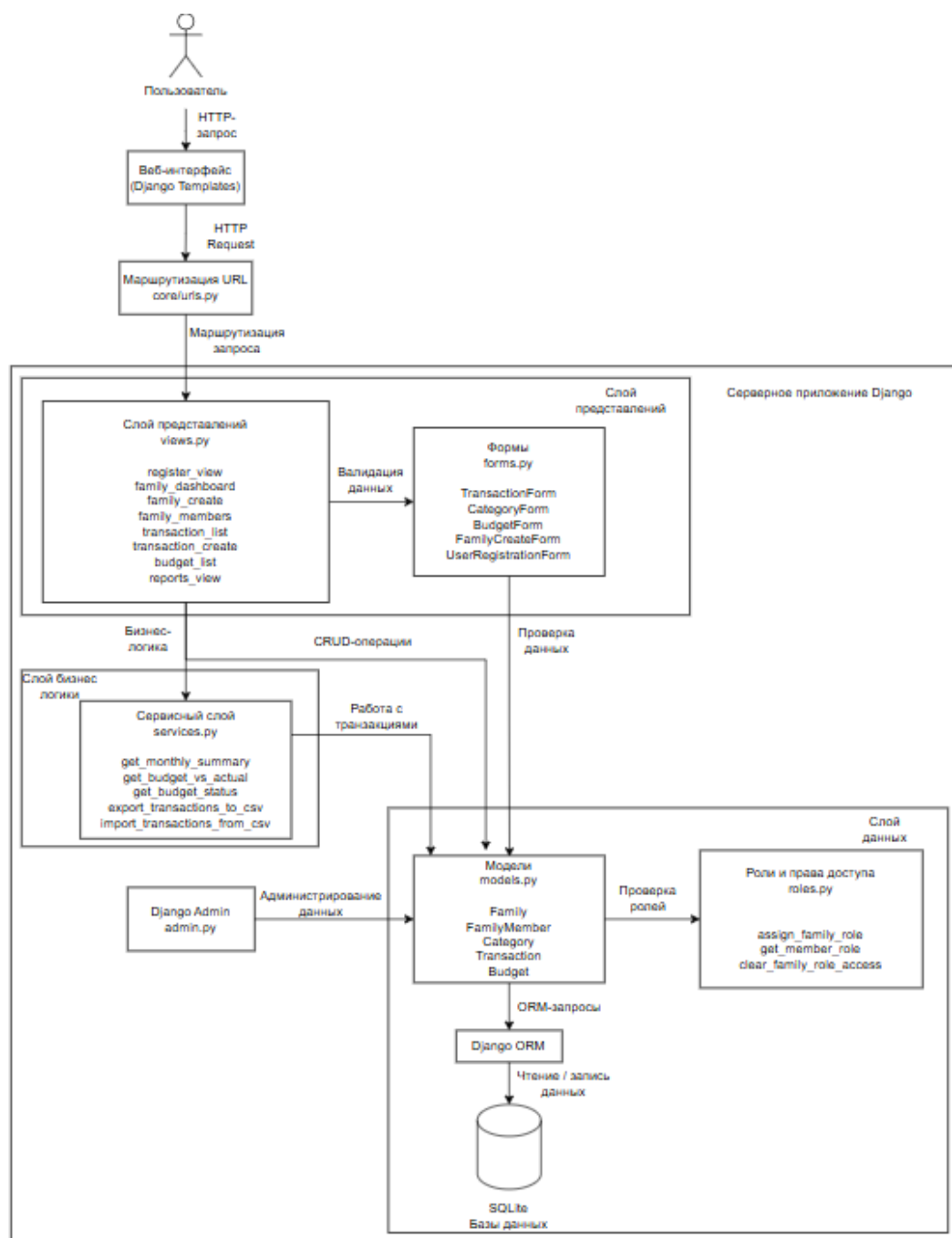


Рисунок 4 – Общая архитектура веб-приложения

Таким образом, выбранная архитектура обеспечивает логическое разделение компонентов системы, упрощает сопровождение программного кода и создает возможность дальнейшего расширения функциональности веб-приложения.

3.2 Использование паттерна MVT

Веб-приложение реализовано с использованием архитектурного паттерна MVT, который применяется во фреймворке Django. Паттерн MVT

включает три основных компонента: Model, View и Template.

Компонент Model отвечает за описание структуры данных и взаимодействие с базой данных. В разрабатываемой системе модели используются для представления основных сущностей предметной области: семейной группы, участника семьи, категории финансовой операции, транзакции, бюджета, комментария и истории изменений. Каждая модель содержит набор полей, определяющих структуру соответствующей таблицы базы данных, а также связи с другими сущностями.

Компонент View отвечает за обработку HTTP-запросов и реализацию бизнес-логики приложения. Представления получают запросы от пользователя, выполняют необходимые проверки, обращаются к моделям данных, формируют результат обработки и передают данные в шаблон или возвращают ответ API. Через представления реализуются функции создания семейной группы, добавления транзакции, редактирования категории, установки бюджетного ограничения и формирования отчетности.

Компонент Template предназначен для формирования HTML-страниц пользовательского интерфейса. Шаблоны получают данные от представлений и отображают их в удобном виде. С помощью шаблонов реализуются страницы регистрации, авторизации, личного кабинета, списка транзакций, управления категориями, просмотра бюджета и финансовой аналитики.

Использование паттерна MVT позволяет разделить структуру данных, бизнес-логику и пользовательский интерфейс. Это делает проект более понятным, упрощает изменение отдельных частей приложения и снижает связанность компонентов.

3.3 Структура проекта

Структура проекта организована по модульному принципу. Основные файлы и каталоги Django-приложения распределены в соответствии с их назначением. Такой подход облегчает сопровождение проекта и позволяет быстро находить необходимые элементы программной системы.

Таблица 6 – Основные элементы структуры проекта

Элемент проекта	Назначение
settings.py	Хранение основных настроек проекта, подключенных приложений, базы данных и параметров безопасности
urls.py	Описание маршрутов веб-приложения и подключение URL-адресов отдельных модулей
models.py	Описание моделей данных и связей между сущностями
views.py	Реализация обработки пользовательских запросов и бизнес-логики
forms.py	Описание форм для ввода и проверки пользовательских данных
serializers.py	Преобразование данных моделей для работы REST API
admin.py	Регистрация моделей в административной панели Django
tests.py	Автоматические тесты для проверки серверной логики
templates	HTML-шаблоны пользовательского интерфейса
migrations	Миграции базы данных, отражающие изменения структуры моделей

Основная логика приложения сосредоточена в моделях, представлениях, формах и сериализаторах. Модели описывают структуру данных, формы отвечают за ввод и валидацию информации, представления выполняют обработку запросов, а сериализаторы используются для формирования API-ответов.

Каталог шаблонов содержит HTML-страницы, которые используются для отображения пользовательского интерфейса. Благодаря механизму наследования шаблонов общие элементы интерфейса, такие как меню навигации, заголовки и блоки сообщений, могут использоваться повторно на разных страницах приложения.

Файлы миграций позволяют фиксировать изменения структуры базы данных. При изменении моделей создаются новые миграции, которые затем применяются к базе данных. Это обеспечивает согласованность между программным кодом и фактической структурой таблиц.

3.4 Организация маршрутизации

Маршрутизация в веб-приложении реализована средствами Django. URL-адреса определяют, какое представление должно обработать конкретный пользовательский запрос. Такой механизм позволяет связать страницы интерфейса и API-endpoint'ы с соответствующими функциями серверной логики.

Основные маршруты приложения связаны с регистрацией и авторизацией пользователей, просмотром главной страницы, управлением семейными группами, категориями, финансовыми транзакциями, бюджетными ограничениями и отчетностью. Также в проекте предусмотрены маршруты для административной панели Django и документации API.

При обращении пользователя к определенному адресу Django анализирует URL-запрос и выбирает соответствующее представление. После этого представление выполняет необходимые действия: получает данные из базы данных, проверяет права доступа, обрабатывает форму или возвращает ответ пользователю.

Например, при переходе на страницу списка транзакций сервер получает текущего авторизованного пользователя, определяет доступные ему финансовые операции и передает данные в HTML-шаблон. При добавлении новой транзакции сервер принимает данные формы, проверяет корректность введенной информации, сохраняет запись в базе данных и перенаправляет пользователя на страницу со списком операций.

Отдельную часть маршрутизации составляют API-маршруты. Они используются для получения и изменения данных в формате JSON. Документация API доступна через Swagger UI, что упрощает проверку endpoint'ов и делает структуру серверного API более наглядной.

3.5 Архитектура системы авторизации и разграничения прав доступа

В разрабатываемом веб-приложении реализована система авторизации и разграничения прав доступа. Она необходима для защиты финансовой информации пользователей и ограничения доступа к данным семейных групп.

Пользователь получает доступ к основным функциям системы только после регистрации и входа в учетную запись. Авторизация позволяет определить, какой пользователь выполняет запрос, какие данные ему доступны и какие действия он может совершать. Для неавторизованных пользователей доступ к страницам управления финансами ограничивается.

В системе выделяются несколько категорий пользователей: владелец семейной группы, участник семейной группы и администратор. Владелец семейной группы имеет расширенные права: он может создавать семейную группу, управлять ее участниками, контролировать общие категории и бюджетные ограничения. Участник семейной группы может работать с общими финансовыми данными в рамках предоставленных прав. Администратор осуществляет техническое сопровождение системы через административную панель Django.

Разграничение доступа особенно важно при работе с семейными группами. Пользователь должен видеть только те финансовые операции, категории и бюджеты, которые относятся к нему лично или к семейной группе, участником которой он является. Такой подход предотвращает доступ к чужим финансовым данным и повышает безопасность системы.

Для защиты данных используются встроенные механизмы Django: система аутентификации, проверка авторизации, защита форм от CSRF-атак, ограничение доступа к административной панели и проверка прав пользователя при выполнении операций.

Таким образом, архитектура авторизации обеспечивает безопасную работу пользователей с финансовой информацией и поддерживает совместное ведение семейного бюджета с разграничением прав.

3.6 Архитектура REST API

В проекте реализован REST API, предназначенный для взаимодействия с данными приложения в формате JSON. API расширяет возможности серверной части и позволяет обращаться к функциям системы не только через HTML-интерфейс, но и через программные запросы.

REST API используется для работы с основными сущностями системы: семейными группами, категориями, финансовыми транзакциями, бюджетными ограничениями, комментариями и историей изменений. Через API можно получать данные, создавать новые записи, изменять существующие объекты и выполнять проверку работы серверной логики.

Для реализации API используется Django REST Framework. Данная библиотека предоставляет инструменты для создания сериализаторов, API-представлений, маршрутов и обработки HTTP-методов. Сериализаторы преобразуют объекты моделей Django в формат JSON и обратно, что позволяет удобно обмениваться данными между клиентом и сервером.

Для документирования API используется Swagger/OpenAPI. Документация формируется автоматически и позволяет просматривать доступные endpoint'ы, параметры запросов, структуру ответов и возможные HTTP-методы. Наличие Swagger UI упрощает тестирование серверной части, так как разработчик может выполнять запросы прямо из браузера.

Использование REST API делает архитектуру приложения более гибкой. В дальнейшем на основе существующего API можно реализовать отдельное frontend-приложение, мобильное приложение или интеграцию с внешними сервисами учета финансов.

3.7 Взаимодействие компонентов системы

Взаимодействие компонентов веб-приложения начинается с действия пользователя в браузере. Пользователь открывает страницу, заполняет форму или выполняет операцию, после чего браузер отправляет HTTP-запрос на сервер Django.

Серверная часть принимает запрос и передает его соответствующему представлению. Представление проверяет авторизацию пользователя, анализирует полученные данные и обращается к моделям. Модели взаимодействуют с базой данных через ORM Django, выполняя операции чтения, создания, изменения или удаления записей.

После обработки запроса сервер формирует ответ. Если запрос был выполнен через веб-интерфейс, данные передаются в HTML-шаблон, который формирует страницу для отображения в браузере. Если запрос был выполнен через API, сервер возвращает ответ в формате JSON.

При работе с финансовыми операциями взаимодействие компонентов можно описать следующим образом. Пользователь открывает форму

добавления транзакции и вводит сумму, категорию, описание и дату операции. После отправки формы представление проверяет корректность данных, определяет пользователя и связанную семейную группу, сохраняет транзакцию в базе данных и обновляет финансовую статистику. Затем пользователь получает страницу с актуальным списком операций.

При работе с бюджетными ограничениями система сравнивает сумму расходов по выбранной категории и периоду с установленным лимитом. Это позволяет пользователю контролировать превышение бюджета и анализировать структуру расходов.

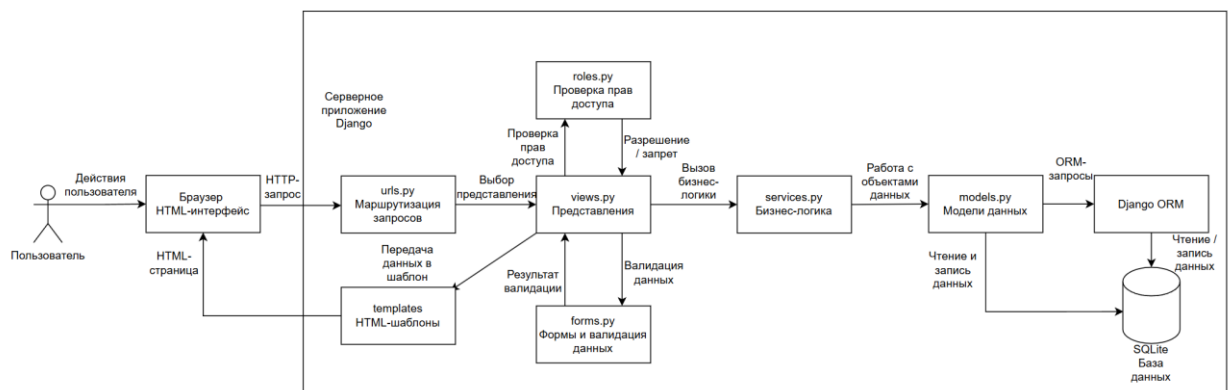


Рисунок 5 – Диаграмма взаимодействия компонентов системы

3.8 Вывод по разделу

В данном разделе была рассмотрена архитектура веб-приложения для планирования семейного бюджета. Система построена на основе клиент-серверной архитектуры и использует паттерн MVT, характерный для фреймворка Django.

Были определены основные компоненты приложения: пользовательский интерфейс, серверная часть, REST API и база данных. Рассмотрена структура проекта, организация маршрутизации, система авторизации и разграничения прав доступа, а также взаимодействие компонентов при выполнении пользовательских операций.

4 Разработка серверной части веб-приложения

4.1 Общая характеристика серверной части

Серверная часть веб-приложения для планирования семейного бюджета реализована с использованием фреймворка Django. Она отвечает за обработку пользовательских запросов, выполнение бизнес-логики, взаимодействие с базой данных, проверку прав доступа и формирование ответов для клиентского слоя.

Основная задача серверной части заключается в обеспечении корректной работы функций учета семейных финансов. Через серверную часть выполняются регистрация и авторизация пользователей, создание семейных групп, управление категориями финансовых операций, добавление доходов и расходов, установка бюджетных ограничений, формирование отчетности, обработка комментариев и фиксация истории изменений.

Веб-приложение построено по принципу разделения ответственности между отдельными компонентами. Модели описывают структуру данных, представления обрабатывают HTTP-запросы, формы выполняют валидацию пользовательского ввода, шаблоны отвечают за отображение данных, а сериализаторы используются для работы REST API.

Серверная часть взаимодействует с базой данных через ORM Django. Это позволяет выполнять операции создания, чтения, изменения и удаления данных без написания SQL-запросов вручную. Такой подход упрощает разработку и снижает вероятность ошибок при работе с базой данных.

Основные серверные модули приложения представлены в таблице 7.

Таблица 7 – Основные серверные модули веб-приложения

Серверный модуль	Назначение
Модуль пользователей	Обеспечивает регистрацию, авторизацию, выход из системы и ограничение доступа к защищенным страницам
Модуль семейных групп	Отвечает за создание семейных групп, хранение участников и разграничение прав внутри группы
Модуль категорий	Используется для создания, редактирования и удаления категорий доходов и расходов
Модуль транзакций	Обеспечивает добавление, изменение, удаление и просмотр финансовых операций

Продолжение таблицы 7

Модуль бюджетов	Позволяет задавать лимиты расходов по категориям и контролировать их выполнение
Модуль отчетности	Формирует сводную информацию о доходах, расходах, балансе и структуре финансовых операций
Модуль импорта и экспорта	Обеспечивает загрузку и выгрузку финансовых данных в формате CSV
Модуль комментариев	Позволяет добавлять пояснения к финансовым записям
Модуль истории изменений	Фиксирует изменение важных данных и действия пользователей
Модуль REST API	Предоставляет программный доступ к данным приложения через HTTP-запросы

4.2 Реализация пользовательской модели и авторизации

Одним из основных элементов серверной части является система пользователей. Пользовательская модель используется для идентификации пользователей, хранения учетных данных и ограничения доступа к финансовой информации.

В приложении предусмотрена регистрация пользователей, вход в систему и выход из учетной записи. После авторизации пользователь получает доступ к личному кабинету и основным функциям системы. Неавторизованные пользователи не могут просматривать финансовые операции, создавать семейные группы, изменять категории и работать с бюджетами.

Для авторизации используются встроенные механизмы Django. Они обеспечивают проверку учетных данных, управление пользовательскими сессиями и защиту страниц, доступных только зарегистрированным пользователям. При попытке открыть защищенную страницу без авторизации пользователь перенаправляется на страницу входа.

Разграничение прав доступа реализуется на уровне представлений и бизнес-логики. Пользователь может работать только с теми данными, которые принадлежат ему лично или относятся к семейной группе, участником которой он является. Такой подход позволяет защитить финансовую информацию от несанкционированного доступа.

Администратор системы имеет доступ к административной панели Django. Через нее можно управлять пользователями, семейными группами, категориями, транзакциями, бюджетами и другими объектами системы.

4.3 Реализация управления семейными группами

Для поддержки совместного ведения бюджета в приложении реализована логика семейных групп. Семейная группа представляет собой общее финансовое пространство, в рамках которого несколько пользователей могут вести учет доходов и расходов.

Владелец семейной группы получает расширенные права на управление общими финансовыми данными. Он может просматривать информацию о группе, управлять участниками, работать с общими категориями и контролировать бюджетные ограничения.

Участник семейной группы получает доступ к совместным финансовым операциям и может участвовать в ведении общего бюджета. При этом система проверяет принадлежность пользователя к группе перед выполнением операций. Это необходимо для того, чтобы пользователь не мог получить доступ к чужой семейной группе.

Для связи пользователей и семейных групп используется отдельная сущность участника семьи. Она хранит информацию о пользователе, семейной группе, дате присоединения и роли пользователя в группе. Такой подход позволяет гибко организовать совместную работу нескольких пользователей и разграничить их права.

Реализация семейных групп позволяет приложению поддерживать не только индивидуальный учет финансов, но и совместное планирование бюджета несколькими участниками.

4.4 Реализация управления категориями финансовых операций

Категории финансовых операций используются для группировки доходов и расходов. Они позволяют структурировать финансовые данные и выполнять последующий анализ бюджета по направлениям затрат или

источникам поступлений.

В серверной части реализованы функции создания, просмотра, изменения и удаления категорий. Каждая категория содержит название, тип операции и связь с пользователем или семейной группой. Тип категории определяет, относится она к доходам или расходам.

Пользователь может создавать личные категории, которые доступны только ему. Также в системе предусмотрены категории, связанные с семейной группой. Такие категории могут использоваться участниками семьи при совместном ведении бюджета.

При создании или изменении категории сервер проверяет корректность введенных данных. Название категории не должно быть пустым, а тип категории должен соответствовать допустимым значениям. Также выполняется проверка прав доступа, чтобы пользователь мог изменять только свои категории или категории своей семейной группы.

Использование категорий упрощает учет финансовых операций и позволяет формировать более точную отчетность по доходам и расходам.

4.5 Реализация управления финансовыми транзакциями

Финансовая транзакция является одной из ключевых сущностей веб-приложения. Она отражает конкретную операцию пользователя: получение дохода или совершение расхода.

В серверной части реализованы функции добавления, просмотра, редактирования и удаления транзакций. При создании транзакции пользователь указывает сумму, категорию, описание и дату операции. Также транзакция связывается с пользователем и при необходимости с семейной группой.

Система поддерживает разделение операций на доходы и расходы. Это позволяет корректно рассчитывать общий баланс, сумму поступлений и сумму затрат за выбранный период. Категория операции используется для более детального анализа структуры бюджета.

При обработке транзакций сервер выполняет проверку данных. Сумма

операции должна быть корректной, категория должна существовать и быть доступной пользователю, а дата операции должна быть указана в допустимом формате. Если данные введены некорректно, пользователь получает сообщение об ошибке.

Для удобства работы с большим количеством операций может использоваться фильтрация транзакций по категории, типу операции, дате и принадлежности к семейной группе. Это упрощает поиск нужных записей и анализ финансовой истории.

Реализация транзакций обеспечивает базовую функциональность приложения, так как именно на основе доходов и расходов формируются отчеты, статистика и контроль бюджетных ограничений.

Основные операции, выполняемые серверной частью при работе с финансовыми транзакциями, представлены в таблице 8.

Таблица 8 – Операции серверной части при обработке финансовых транзакций

Операция	Описание обработки на сервере
Создание транзакции	Сервер принимает данные формы, проверяет сумму, дату, категорию и принадлежность операции пользователю или семейной группе
Просмотр транзакций	Система получает из базы данных только те операции, которые доступны текущему пользователю
Редактирование транзакции	Сервер проверяет права доступа, корректность новых данных и обновляет запись в базе данных
Удаление транзакции	Перед удалением выполняется проверка, имеет ли пользователь право изменять выбранную финансовую операцию
Фильтрация операций	Транзакции могут отбираться по типу, категории, дате, пользователю или семейной группе
Расчет итогов	На основе транзакций вычисляются суммы доходов, расходов и текущий баланс
Обновление отчетности	После изменения финансовых операций данные используются при формировании аналитики и отчетов
Контроль бюджета	Расходные операции учитываются при проверке превышения установленных бюджетных ограничений

4.6 Реализация бюджетных ограничений

Бюджетные ограничения предназначены для контроля расходов пользователя или семейной группы. Они позволяют установить лимит затрат

по определенной категории за заданный период времени.

В серверной части реализована возможность создания, просмотра, изменения и удаления бюджетов. При создании бюджетного ограничения пользователь указывает категорию, сумму лимита, период действия и принадлежность бюджета к пользователю или семейной группе.

После добавления финансовых операций система может сравнивать фактические расходы с установленным лимитом. Если сумма расходов по категории приближается к ограничению или превышает его, пользователь получает возможность увидеть это при просмотре бюджета и отчетности.

Использование бюджетных ограничений помогает пользователям контролировать расходы и планировать финансовое поведение.

При работе с бюджетами сервер проверяет права доступа и корректность данных. Пользователь может управлять только теми бюджетами, которые принадлежат ему лично или относятся к семейной группе, участником которой он является.

4.7 Реализация отчетности и аналитики

Для анализа финансового состояния в приложении реализованы функции отчетности и аналитики. Они позволяют пользователю получать сводную информацию о доходах, расходах, балансе и структуре финансовых операций.

Отчетность формируется на основе сохраненных транзакций. Серверная часть получает данные из базы данных, группирует их по категориям, типам операций и периодам, после чего передает результат в шаблон или API-ответ.

В системе может отображаться общая сумма доходов, общая сумма расходов, текущий баланс, расходы по категориям и информация о выполнении бюджетных ограничений. Это позволяет пользователю оценить финансовое состояние и выявить основные направления затрат.

Аналитика особенно важна при совместном ведении семейного бюджета. Участники семейной группы могут видеть общую картину доходов и расходов, анализировать структуру затрат и принимать решения о

перераспределении бюджета.

Формирование отчетности выполняется на серверной стороне, что позволяет централизованно обрабатывать данные и обеспечивать единообразие расчетов для всех пользователей системы.

4.8 Реализация импорта и экспорта данных

Для повышения удобства работы с финансовыми данными в системе предусмотрены функции импорта и экспорта данных. Они позволяют переносить информацию между приложением и внешними файлами.

Экспорт данных может использоваться для сохранения финансовой истории в отдельный файл. Пользователь получает возможность выгрузить сведения о транзакциях, категориях или других объектах системы для дальнейшего анализа или резервного хранения.

Импорт данных позволяет загрузить финансовые операции из внешнего файла. Это удобно в случае переноса информации из другой системы учета бюджета или восстановления ранее сохраненных данных.

При обработке импортируемых данных сервер выполняет проверку структуры файла и корректности значений. Если данные содержат ошибки, система должна предотвращать некорректную запись в базу данных. Такой подход позволяет сохранить целостность информации и избежать появления неверных финансовых операций.

Реализация импорта и экспорта расширяет функциональность приложения и делает его более удобным для практического использования.

4.9 Реализация комментариев и истории изменений

В серверной части приложения реализованы дополнительные механизмы комментариев и истории изменений. Они предназначены для повышения прозрачности работы с данными и удобства сопровождения финансовых записей.

Комментарии позволяют добавлять пояснения к объектам системы. Например, пользователь может оставить примечание к финансовой операции,

пояснив назначение расхода или источник дохода. Это особенно полезно при совместном ведении семейного бюджета, когда нескольким участникам необходимо понимать причину появления той или иной записи.

История изменений предназначена для фиксации важных действий пользователей. В системе может сохраняться информация о том, кто изменил запись, какое поле было изменено, какое значение было раньше и какое стало после изменения. Такой механизм позволяет отслеживать изменения финансовых данных и повышает надежность работы системы.

Фиксация истории особенно важна для операций, связанных с изменением суммы транзакции, категории, даты, бюджетного ограничения или принадлежности записи к семейной группе. Благодаря этому можно восстановить последовательность изменений и определить, какие действия были выполнены пользователями.

Использование комментариев и истории изменений делает приложение более информативным и удобным при совместной работе с семейным бюджетом.

4.10 Реализация REST API и документации Swagger/OpenAPI

Помимо HTML-интерфейса, серверная часть приложения предоставляет REST API. Он используется для программного взаимодействия с данными системы и позволяет получать, создавать, изменять и удалять объекты через HTTP-запросы.

REST API реализован с использованием Django REST Framework. Для преобразования объектов моделей в формат JSON используются сериализаторы. Они определяют, какие поля модели будут передаваться в API-ответах и какие данные могут быть получены от клиента.

Через API могут быть доступны операции работы с семейными группами, категориями, транзакциями, бюджетами, комментариями и историей изменений. Использование API делает приложение более гибким и позволяет в дальнейшем подключить к нему отдельный frontend или мобильное приложение.

Для документирования API используется Swagger/OpenAPI. Автоматически сформированная документация позволяет просматривать список endpoint'ов, доступные HTTP-методы, параметры запросов и структуру ответов. Swagger UI также дает возможность выполнять тестовые запросы через браузер.

Наличие документации API упрощает тестирование серверной части и делает проект более понятным для разработчика, проверяющего или потенциального пользователя системы.

Основные группы endpoint'ов REST API представлены в таблице 9.

Таблица 9 – Основные группы endpoint'ов REST API

Группа endpoint'ов	Назначение
/api/families/	Получение и управление семейными группами
/api/categories/	Работа с категориями доходов и расходов
/api/transactions/	Создание, просмотр, изменение и удаление финансовых транзакций
/api/budgets/	Управление бюджетными ограничениями
/api/comments/	Работа с комментариями к объектам системы
/api/history/	Получение истории изменений данных
/api/schema/	Получение OpenAPI-схемы серверного API
/api/docs/	Просмотр Swagger UI для тестирования и изучения API

4.11 Вывод по разделу

В данном разделе была рассмотрена реализация серверной части веб-приложения для планирования семейного бюджета. Серверная часть построена на основе фреймворка Django и обеспечивает обработку HTTP-запросов, выполнение бизнес-логики, взаимодействие с базой данных и проверку прав доступа.

Были описаны основные функциональные модули системы: пользовательская модель и авторизация, семейные группы, категории финансовых операций, транзакции, бюджетные ограничения, отчетность, импорт и экспорт данных, комментарии, история изменений и REST API.

5 Реализация базы данных

5.1 Общая характеристика базы данных

Для хранения данных веб-приложения планирования семейного бюджета используется реляционная база данных SQLite. Данная система управления базами данных выбрана в связи с простотой настройки, отсутствием необходимости развёртывания отдельного сервера и полной поддержкой со стороны фреймворка Django.

База данных предназначена для хранения информации о пользователях, семейных группах, участниках семей, категориях финансовых операций, транзакциях, бюджетных ограничениях, комментариях и истории изменений. Использование реляционной модели позволяет организовать данные в виде взаимосвязанных таблиц и обеспечить целостность информации за счёт внешних ключей.

Взаимодействие серверной части с базой данных осуществляется через Django ORM. Такой подход позволяет описывать структуру таблиц в виде Python-классов, а операции создания, чтения, изменения и удаления данных выполнять без прямого написания SQL-запросов. Это упрощает разработку, снижает вероятность ошибок и делает программный код более понятным.

База данных играет ключевую роль в работе приложения, так как все основные функции системы связаны с сохранением и обработкой финансовой информации. Через неё обеспечивается хранение учетных записей пользователей, объединение пользователей в семейные группы, учет доходов и расходов, контроль лимитов бюджета и формирование аналитической информации.

5.2 Графическое представление базы данных

Для наглядного представления структуры базы данных была разработана ER-диаграмма. Она отражает основные сущности системы, их атрибуты и взаимосвязи между таблицами.

ER-диаграмма позволяет показать, каким образом пользователи связаны

с семейными группами, категориями, транзакциями и бюджетными ограничениями. Также на диаграмме отображаются дополнительные сущности, используемые для комментариев и фиксации истории изменений.

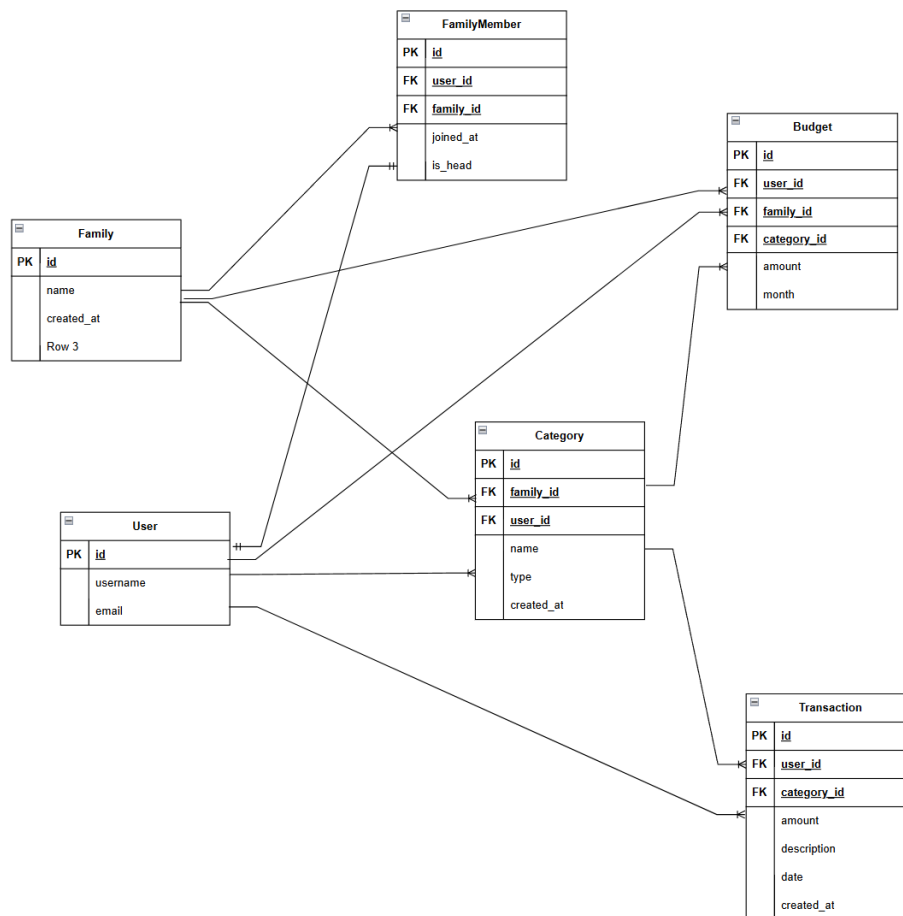


Рисунок 6 – ER-диаграмма базы данных веб-приложения

На диаграмме представлены основные таблицы базы данных: пользователь, семейная группа, участник семьи, категория, транзакция, бюджет, комментарий и история изменений. Связи между таблицами реализуются с помощью внешних ключей, что обеспечивает согласованность данных и предотвращает появление записей, не связанных с основными объектами системы.

Использование ER-диаграммы упрощает понимание логики хранения данных и позволяет перейти от предметной области к программной реализации моделей Django.

5.3 Основные сущности базы данных

Основные сущности базы данных соответствуют ключевым объектам предметной области. Они обеспечивают хранение данных, необходимых для работы системы управления семейным бюджетом.

Таблица 10 – Основные сущности базы данных

Сущность	Назначение
Пользователь	Хранение учетных данных и идентификация пользователя в системе
Семейная группа	Объединение нескольких пользователей для совместного ведения бюджета
Участник семьи	Связь пользователя с семейной группой и хранение его роли
Категория	Группировка доходов и расходов по типам финансовых операций
Транзакция	Хранение финансовой операции: дохода или расхода
Бюджет	Хранение лимита расходов по категории за определенный период
Комментарий	Добавление пояснений к объектам системы или финансовым операциям
История изменений	Фиксация изменений важных данных и действий пользователей

Выделение данных сущностей позволяет охватить основные сценарии работы приложения: регистрацию пользователя, создание семейной группы, добавление финансовых операций, анализ расходов, контроль бюджета и сопровождение данных.

5.4 Описание сущностей пользователей и семейных групп

Одной из основных сущностей базы данных является пользователь. Пользователь используется для авторизации в системе и связывается с финансовыми данными, которые он создает или просматривает. Для каждого пользователя хранятся учетные данные, необходимые для входа в систему и идентификации при выполнении операций.

Семейная группа предназначена для организации совместного ведения бюджета. Она объединяет нескольких пользователей в рамках одного финансового пространства. Благодаря этой сущности пользователи могут вести не только личный учет, но и совместно работать с общими категориями, транзакциями и бюджетными ограничениями.

Связь между пользователем и семейной группой реализуется через сущность участника семьи. Такая промежуточная сущность необходима, так как один пользователь может быть связан с семейной группой, а семейная группа может включать несколько пользователей. В сущности участника семьи хранится информация о пользователе, семейной группе, дате присоединения и роли участника.

Таблица 11 – Сущности пользователей и семейных групп

Сущность	Основные поля	Описание
Пользователь	id, username, password, email	Хранит данные учетной записи пользователя
Семейная группа	id, name, created_at	Описывает общее финансовое пространство семьи
Участник семьи	id, user_id, family_id, joined_at, is_head	Связывает пользователя с семьей и определяет его статус

Использование отдельной сущности участника семьи позволяет реализовать разграничение прав доступа. Например, владелец семейной группы может иметь расширенные права по управлению общими данными, а обычный участник — доступ только к просмотру и добавлению финансовых операций.

5.5 Описание сущностей финансового учета

Основную часть базы данных составляют сущности, связанные с финансовым учетом. К ним относятся категории, транзакции и бюджетные ограничения.

Категории используются для классификации финансовых операций. Каждая категория имеет название и тип, определяющий принадлежность к доходам или расходам. Категории позволяют группировать операции и формировать отчетность по направлениям затрат или источникам поступлений.

Транзакция представляет собой отдельную финансовую операцию. Она содержит сумму, дату, описание, связь с пользователем и выбранной категорией. Транзакции являются основой для расчета общего баланса, суммы доходов, суммы расходов и финансовой аналитики.

Бюджет используется для хранения лимитов расходов по отдельным категориям. Он позволяет задавать ограничение на определенный период и контролировать фактические расходы. При формировании отчетности данные бюджета сравниваются с суммой транзакций по соответствующей категории.

Таблица 12 – Сущности финансового учета

Сущность	Основные поля	Описание
Категория	id, name, type, user_id, family_id, created_at	Используется для группировки доходов и расходов
Транзакция	id, user_id, category_id, amount, description, date, created_at	Хранит сведения о финансовой операции
Бюджет	id, user_id, category_id, limit_amount, period_start, period_end	Хранит бюджетное ограничение по категории и периоду

Связь транзакции с категорией позволяет анализировать структуру расходов. Связь с пользователем позволяет определить автора операции и ограничить доступ к данным.

5.6 Описание сущностей контроля и сопровождения данных

Для повышения прозрачности работы системы в базе данных используются дополнительные сущности комментариев и истории изменений.

Комментарии позволяют пользователям добавлять пояснения к финансовым записям. Это может быть полезно при совместном ведении бюджета, когда участникам семьи необходимо понимать назначение конкретной операции или причину изменения данных.

История изменений используется для фиксации действий пользователей. Она может хранить сведения о том, какая запись была изменена, какое поле было затронуто, каким было старое значение и какое новое значение было установлено.

Таблица 13 – Сущности контроля и сопровождения данных

Сущность	Основные поля	Описание
Комментарий	id, user_id, text, created_at	Хранит пояснения пользователей к объектам системы
История изменений	id, user_id, object_type, object_id, field_name, old_value, new_value, changed_at	Фиксирует изменения важных данных

Наличие данных сущностей особенно важно для совместной работы нескольких пользователей. Комментарии повышают информативность записей, а история изменений позволяет контролировать корректность внесенных правок.

5.7 Связи между сущностями

Связи между сущностями базы данных реализуются с помощью внешних ключей. Это позволяет обеспечить целостность данных и корректно организовать взаимодействие между объектами системы.

Пользователь может создавать собственные категории, транзакции и бюджеты. Также пользователь может быть участником семейной группы. Семейная группа объединяет нескольких пользователей через сущность участника семьи. Категории могут быть связаны как с отдельным пользователем, так и с семейной группой, что позволяет поддерживать личный и совместный учет финансов.

Транзакция связана с пользователем и категорией. Благодаря этому можно определить автора операции и тип финансовой операции. Бюджет связан с пользователем и категорией, что позволяет контролировать расходы по конкретному направлению. Комментарии и история изменений связываются с пользователем, который выполнил соответствующее действие.

Такая структура связей позволяет обеспечить логичное хранение данных и выполнять выборку информации в зависимости от пользователя, семейной группы, категории или периода.

5.8 Использование Django ORM и миграций

Для работы с базой данных используется Django ORM. ORM позволяет описывать сущности базы данных в виде моделей Python. Каждая модель соответствует отдельной таблице базы данных, а поля модели соответствуют столбцам таблицы.

Использование ORM упрощает выполнение операций с данными. Например, создание транзакции, получение списка категорий, фильтрация

операций по пользователю или подсчет расходов по категории могут выполняться средствами Python-кода без написания SQL-запросов вручную.

Важной частью работы с базой данных являются миграции. Миграции фиксируют изменения структуры моделей и позволяют синхронизировать программный код с фактической структурой базы данных. При добавлении новой модели или изменении полей создается файл миграции, после чего изменения применяются к базе данных.

В проекте миграции используются для создания таблиц пользователей, семейных групп, участников семьи, категорий, транзакций, бюджетов, комментариев и истории изменений. Это обеспечивает последовательное развитие структуры базы данных и позволяет контролировать изменения на разных этапах разработки.

Использование Django ORM и миграций повышает надежность разработки, так как структура базы данных остается связанной с моделями приложения и может быть восстановлена при переносе проекта на другое окружение.

5.9 Обеспечение целостности данных

Целостность данных является важным требованием для веб-приложения планирования семейного бюджета, так как система работает с финансовой информацией пользователей. Ошибки в связях между таблицами или некорректные значения могут привести к неправильному расчету бюджета и отчетности.

Для обеспечения целостности данных используются внешние ключи, ограничения моделей Django и серверная валидация. Внешние ключи позволяют связать транзакции с пользователями и категориями, бюджеты с категориями, а участников семьи с пользователями и семейными группами. Это предотвращает появление записей, которые не имеют связи с основными объектами системы.

На уровне серверной логики выполняется проверка пользовательского ввода. Например, сумма транзакции должна быть положительным числом,

категория должна существовать и быть доступной пользователю, а бюджетное ограничение должно иметь корректный период действия.

Также целостность данных поддерживается за счет разграничения прав доступа. Пользователь может изменять только те записи, которые относятся к нему лично или к семейной группе, участником которой он является. Это предотвращает случайное или несанкционированное изменение чужих финансовых данных.

Таким образом, обеспечение целостности данных реализуется на нескольких уровнях: на уровне структуры базы данных, моделей Django, серверной проверки и контроля доступа пользователей.

5.10 Вывод по разделу

В данном разделе была рассмотрена логика базы данных веб-приложения для планирования семейного бюджета. Для хранения данных используется реляционная база данных SQLite, взаимодействие с которой осуществляется через Django ORM.

Были определены основные сущности базы данных: пользователь, семейная группа, участник семьи, категория, транзакция, бюджет, комментарий и история изменений. Рассмотрены их назначение, основные поля и связи между таблицами.

Разработанная структура базы данных обеспечивает хранение учетных данных пользователей, поддержку совместного ведения семейного бюджета, учет доходов и расходов, контроль бюджетных ограничений, добавление комментариев и фиксацию истории изменений.

Использование внешних ключей, миграций Django и серверной валидации позволяет обеспечить целостность и согласованность данных. Такая структура базы данных соответствует требованиям разрабатываемого веб-приложения и создает основу для надежной работы серверной части системы.

6 Разработка клиентского слоя веб-приложения

6.1 Общая характеристика клиентского слоя

Клиентский слой веб-приложения реализован с использованием Django Templates. Пользователь взаимодействует с системой через веб-браузер. Отдельное frontend-приложение не используется, так как HTML-страницы формируются на стороне сервера.

Клиентский слой обеспечивает:

- регистрацию и авторизацию пользователя;
- переход между разделами приложения;
- просмотр финансовых операций;
- добавление, изменение и удаление транзакций;
- управление категориями доходов и расходов;
- работу с бюджетными ограничениями;
- просмотр отчетности и аналитики;
- взаимодействие с административной панелью Django.

Такой подход упрощает структуру проекта и позволяет связать пользовательский интерфейс напрямую с серверной логикой Django.

6.2 Структура пользовательского интерфейса

Пользовательский интерфейс состоит из набора страниц, каждая из которых отвечает за отдельную функцию системы.

Таблица 14 – Основные страницы пользовательского интерфейса

Страница	Назначение
Страница регистрации	Создание новой учетной записи пользователя
Страница входа	Авторизация пользователя в системе
Главная страница	Переход к основным разделам веб-приложения
Личный кабинет	Просмотр информации о пользователе
Страница семейной группы	Просмотр и управление семейной группой
Страница категорий	Создание и редактирование категорий доходов и расходов
Страница транзакций	Просмотр списка финансовых операций
Форма транзакции	Добавление или изменение дохода/расхода
Страница бюджетов	Управление лимитами расходов
Страница отчетности	Просмотр сводной информации о доходах, расходах и балансе
Административная панель	Управление данными через интерфейс Django Admin

Интерфейс построен так, чтобы пользователь мог быстро перейти к основным действиям: добавить операцию, посмотреть список расходов, изменить категорию или проверить состояние бюджета.

6.3 Реализация шаблонов Django

HTML-страницы реализованы с использованием механизма Django Templates. Шаблоны получают данные от представлений и отображают их пользователю.

В проекте используется общий базовый шаблон, в котором размещаются повторяющиеся элементы интерфейса: заголовок страницы, навигация, блок сообщений и основная область содержимого. Остальные страницы наследуются от базового шаблона и переопределяют только нужные блоки.

Такой подход уменьшает дублирование HTML-кода и упрощает изменение внешнего вида приложения. Например, изменение меню навигации в базовом шаблоне автоматически применяется ко всем страницам.

В шаблонах используются стандартные возможности Django:

- вывод переменных;
- циклы для отображения списков;
- условия для проверки роли или состояния пользователя;
- подключение форм;
- вывод сообщений об ошибках и успешных действиях;
- наследование шаблонов.

6.4 Навигация веб-приложения

Навигация обеспечивает переход между основными разделами системы. Для авторизованного пользователя доступны страницы финансовых операций, категорий, бюджетов, семейной группы и отчетности.

Для неавторизованного пользователя доступны только страницы регистрации и входа. После успешной авторизации пользователь получает доступ к защищенным разделам приложения.

Таблица 15 – Основные элементы навигации

Раздел	Назначение
Главная	Переход к основным функциям системы
Транзакции	Просмотр и управление финансовыми операциями
Категории	Управление категориями доходов и расходов
Бюджеты	Настройка лимитов расходов
Отчеты	Просмотр финансовой аналитики
Семья	Управление семейной группой
Профиль	Просмотр данных учетной записи
Выход	Завершение пользовательской сессии

Навигация строится с учетом авторизации пользователя. Это предотвращает доступ к разделам, требующим входа в систему.

6.5 Вывод по разделу

Клиентский слой веб-приложения реализован средствами Django Templates. Пользовательский интерфейс включает страницы регистрации, авторизации, управления транзакциями, категориями, бюджетами, семейной группой и отчетностью.

Использование серверных шаблонов позволило упростить архитектуру проекта и связать интерфейс напрямую с серверной логикой Django. Реализованные страницы обеспечивают выполнение основных пользовательских сценариев системы планирования семейного бюджета.

7 Тестирование веб-приложения

7.1 Цели и организация тестирования

Тестирование веб-приложения выполнялось для подтверждения корректности основных пользовательских сценариев, устойчивости серверной логики, работоспособности веб-интерфейса, корректности разграничения прав доступа и соответствия реализованного функционала требованиям курсовой работы. Проверка охватывала регистрацию и авторизацию пользователей, создание семейной группы, управление участниками, работу с категориями доходов и расходов, добавление финансовых транзакций, настройку бюджетных ограничений, формирование отчетности, импорт и экспорт данных в формате CSV, а также доступность административного контура.

Так как приложение работает с финансовой информацией, особое внимание при тестировании уделялось корректности расчетов, защите пользовательских данных и проверке прав доступа. Пользователь должен иметь доступ только к собственным финансовым данным или данным семейной группы, участником которой он является. Владелец семейной группы должен иметь расширенные права на управление участниками и общими финансовыми данными, а обычный участник должен быть ограничен в административных действиях внутри группы.

При тестировании использовались автоматизированные тесты, ручная проверка пользовательского интерфейса, проверка документации API через Swagger UI, тестирование запросов через Postman, а также анализ покрытия кода средствами coverage. Такой подход позволяет проверить систему как с точки зрения внутренней бизнес-логики, так и с точки зрения внешнего взаимодействия пользователя с веб-приложением.

7.2 Модульное и интеграционное тестирование

Модульное тестирование направлено на проверку отдельных компонентов серверной части приложения: моделей, сервисных функций, форм, механизмов расчета бюджета, импорта и экспорта данных, а также

правил валидации. Проверялась корректность создания семейных групп, категорий, транзакций и бюджетных ограничений. Также проверялись ограничения, связанные с принадлежностью данных пользователю или семейной группе.

Интеграционное тестирование позволяет убедиться, что несколько компонентов системы корректно взаимодействуют между собой. Например, пользователь проходит регистрацию и авторизацию, создает семейную группу, добавляет категории, вносит доходы и расходы, устанавливает бюджетные ограничения и получает итоговую финансовую статистику. Также проверяется корректность перенаправлений после выполнения операций и недоступность защищенных страниц для неавторизованных пользователей.

Результаты интеграционного тестирования представлены в таблице 16.

Таблица 16 – Результаты интеграционного тестирования

№	Проверяемый сценарий	Проверяемые компоненты	Ожидаемый результат	Результат
1	Регистрация пользователя	Представление регистрации, форма регистрации, модель пользователя	Пользователь успешно создается в системе	Пройдено
2	Создание семейной группы	Представление создания группы, модель Family, модель FamilyMember	Семейная группа создается и связывается с пользователем	Пройдено
3	Создание категории расходов	Представление создания категории, форма категории, модель Category	Категория сохраняется в базе данных	Пройдено
4	Создание финансовой транзакции	Представление создания транзакции, форма транзакции, модели Transaction и Category	Транзакция создается и связывается с выбранной категорией	Пройдено
5	Экспорт и импорт CSV	Представления импорта и экспорта, обработка файла, модели Category и Transaction	Данные корректно выгружаются и загружаются обратно в систему	Пройдено

Для запуска автоматизированных тестов используется команда: `pytest`.

Результаты автоматизированного тестирования подтверждают корректную работу основных сценариев приложения. В тестах проверяются регистрация пользователя, защита страниц от неавторизованного доступа, создание семейной группы, отображение роли участника, список транзакций,

создание транзакции с предупреждением о превышении бюджета, расчет месячной сводки, проверка статуса бюджета, импорт и экспорт CSV-файлов.

```
PS C:\Users\relax\Documents\kursach_6_backend> python -m pytest tests\ -q
plugins: Faker-4.0.12.0, cov-4.1.0, django-4.12.0, env-1.6.0, lazy-fixture-0.6.3, pythonpath-0.7.3, xdist-3.8.0
collected 101 items

tests\test_coverage_completeness.py ..... [ 35%]
tests\test_forms.py ..... [ 40%]
tests\test_integration.py .. [ 42%]
tests\test_models.py ..... [ 52%]
tests\test_permissions.py ..... [ 70%]
tests\test_services.py ..... [ 77%]
tests\test_views.py ..... [ 98%]
tests\test_openapi.py .. [100%]

----- coverage: platform win32, python 3.12.10-final-0 -----
Name                                                    Stmts  Miss  Cover   Missing
-----
family_finance\core\__init__.py                          0      0  100%
family_finance\core\admin.py                             35      0  100%
family_finance\core\apps.py                              4      0  100%
family_finance\core\context_processors.py                 8      0  100%
family_finance\core\forms.py                             181     0  100%
family_finance\core\migrations\0001_initial.py            8      0  100%
family_finance\core\migrations\0002_family_alter_budget_unique_together_and_more.py  7      0  100%
family_finance\core\migrations\0003_alter_budget_options_alter_family_options_and_more.py  6      0  100%
family_finance\core\migrations\0004_create_default_groups.py  19     0  100%
family_finance\core\migrations\0005_auto_20260328_1451.py  4      0  100%
family_finance\core\migrations\0006_add_data_integrity_constraints.py  5      0  100%
family_finance\core\migrations\__init__.py                0      0  100%
family_finance\core\models.py                           116     0  100%
family_finance\core\openapi.py                           47     0  100%
family_finance\core\roles.py                             58     0  100%
family_finance\core\services.py                          144     0  100%
family_finance\core\urls.py                              4      0  100%
family_finance\core\views.py                             362     0  100%
-----
TOTAL                                                    1008     0  100%

Required test coverage of 100% reached. Total coverage: 100.00%
===== 101 passed in 82.83s (0:01:22) =====
```

Рисунок 7 – Результат выполнения автоматизированных тестов

7.3 Тестирование веб-интерфейса

Ручное тестирование веб-интерфейса выполнялось для проверки основных пользовательских сценариев веб-приложения. Основное внимание уделялось тем страницам, через которые пользователь выполняет ключевые действия: регистрацию, авторизацию, работу с финансовыми операциями, настройку бюджетных ограничений и просмотр отчетности.

Первым делом, проверялась работа с финансовыми операциями. Пользователь добавлял доходы и расходы, указывал сумму, категорию, описание и дату операции. После сохранения проверялось отображение новой записи в общем списке транзакций, корректность вывода данных и возможность дальнейшего просмотра финансовой истории.

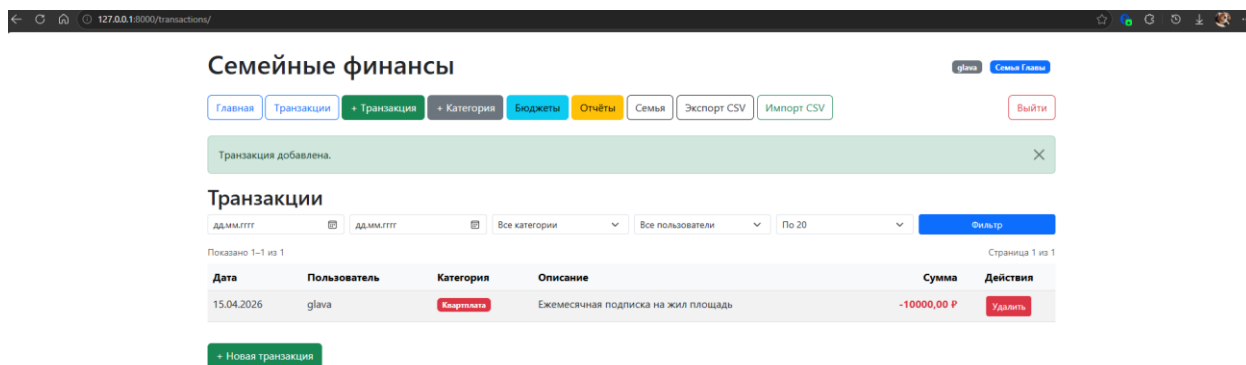


Рисунок 8 – Проверка управления финансовыми транзакциями

Отдельно тестировалась работа с бюджетными ограничениями. Пользователь создавал лимит расходов по выбранной категории и проверял, как система учитывает его при добавлении расходных операций. При превышении установленного лимита проверялось отображение соответствующего предупреждения или изменения состояния бюджета.

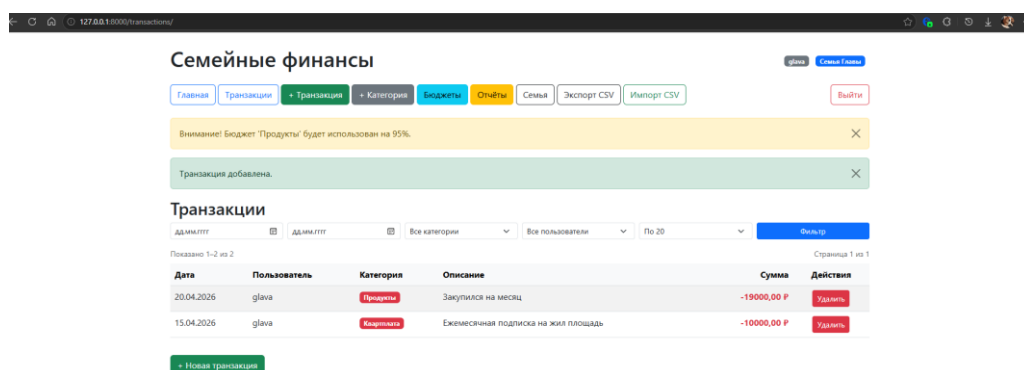


Рисунок 9 – Проверка бюджетных ограничений

Также была проверена страница отчетности. В ходе тестирования контролировалось отображение общей суммы доходов, расходов, текущего баланса и сводной информации по финансовым операциям. Это позволило убедиться, что данные, внесенные пользователем, корректно используются при формировании аналитики.

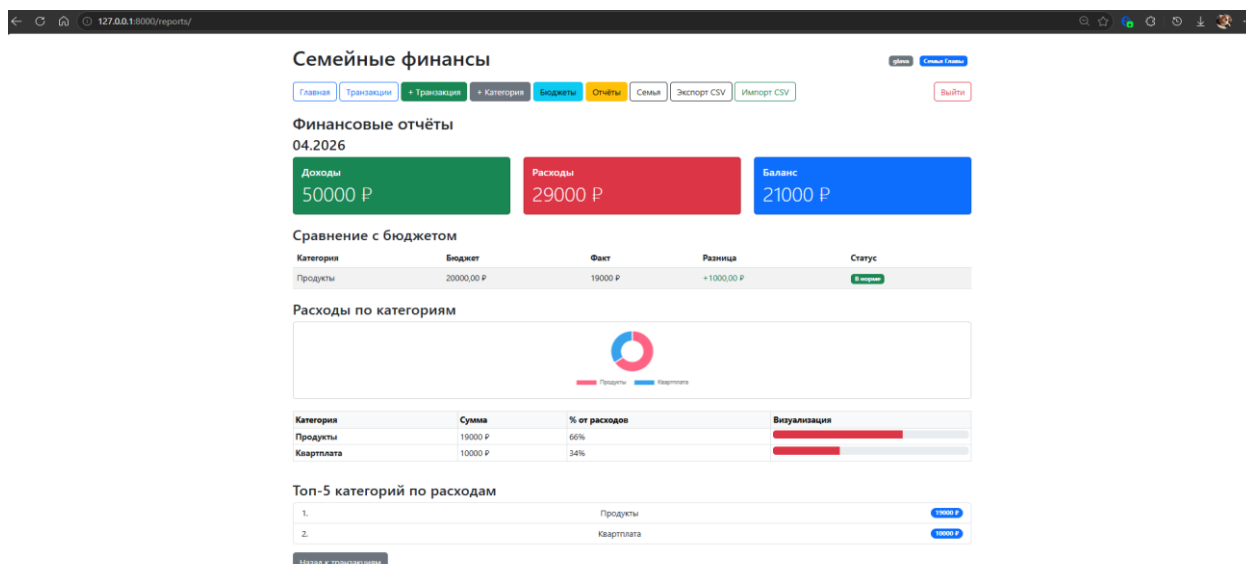


Рисунок 10 – Проверка формирования финансовой отчетности

После выбора CSV-файла система выполняет загрузку данных о финансовых транзакциях и проверяет корректность их структуры. В процессе тестирования была проверена возможность импорта операций с указанием суммы, категории, даты и описания. При успешной обработке файла записи сохраняются в базе данных и становятся доступными пользователю в общем списке транзакций.

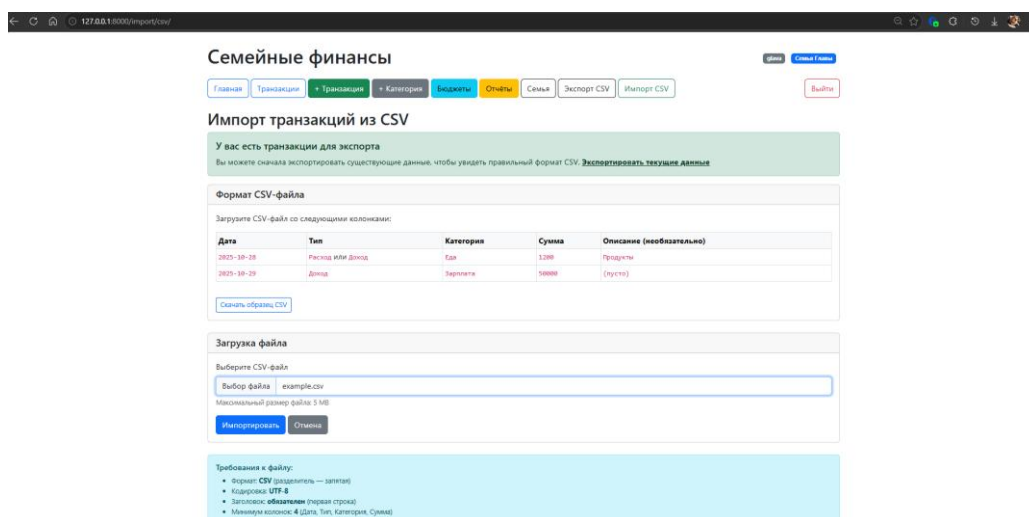


Рисунок 11 – Импорт транзакций из CSV

Таким образом, ручное тестирование веб-интерфейса подтвердило корректную работу основных страниц приложения. Проверенные сценарии охватывают базовый цикл использования системы. Это позволяет сделать вывод, что интерфейс веб-приложения обеспечивает выполнение ключевых функций системы планирования семейного бюджета.

7.4 Тестирование API через Swagger UI

Отдельно проверялась документация API, так как она используется для изучения доступных маршрутов и проверки серверной части приложения. В проекте реализована OpenAPI-схема и интерфейс Swagger UI. Через Swagger UI можно просмотреть доступные группы маршрутов, параметры запросов, типы ответов и возможные коды состояния.

В OpenAPI-схеме описаны механизмы session-аутентификации и CSRF-защиты, что важно для проверки защищенных HTML-форм и POST-запросов.

В ходе тестирования через Swagger UI проверялась доступность страницы документации, загрузка OpenAPI-схемы, наличие основных маршрутов приложения и корректное отображение описания endpoint-ов. Такая проверка подтверждает, что документация API формируется корректно и может использоваться для тестирования и сопровождения серверной части.

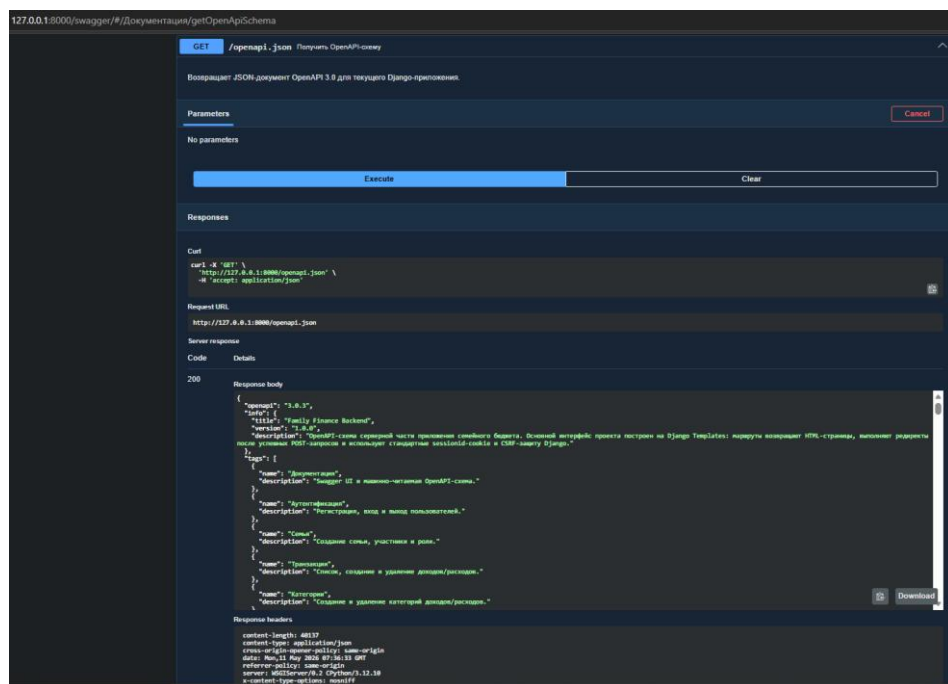


Рисунок 12 – Проверка OpenAPI-схемы через Swagger UI

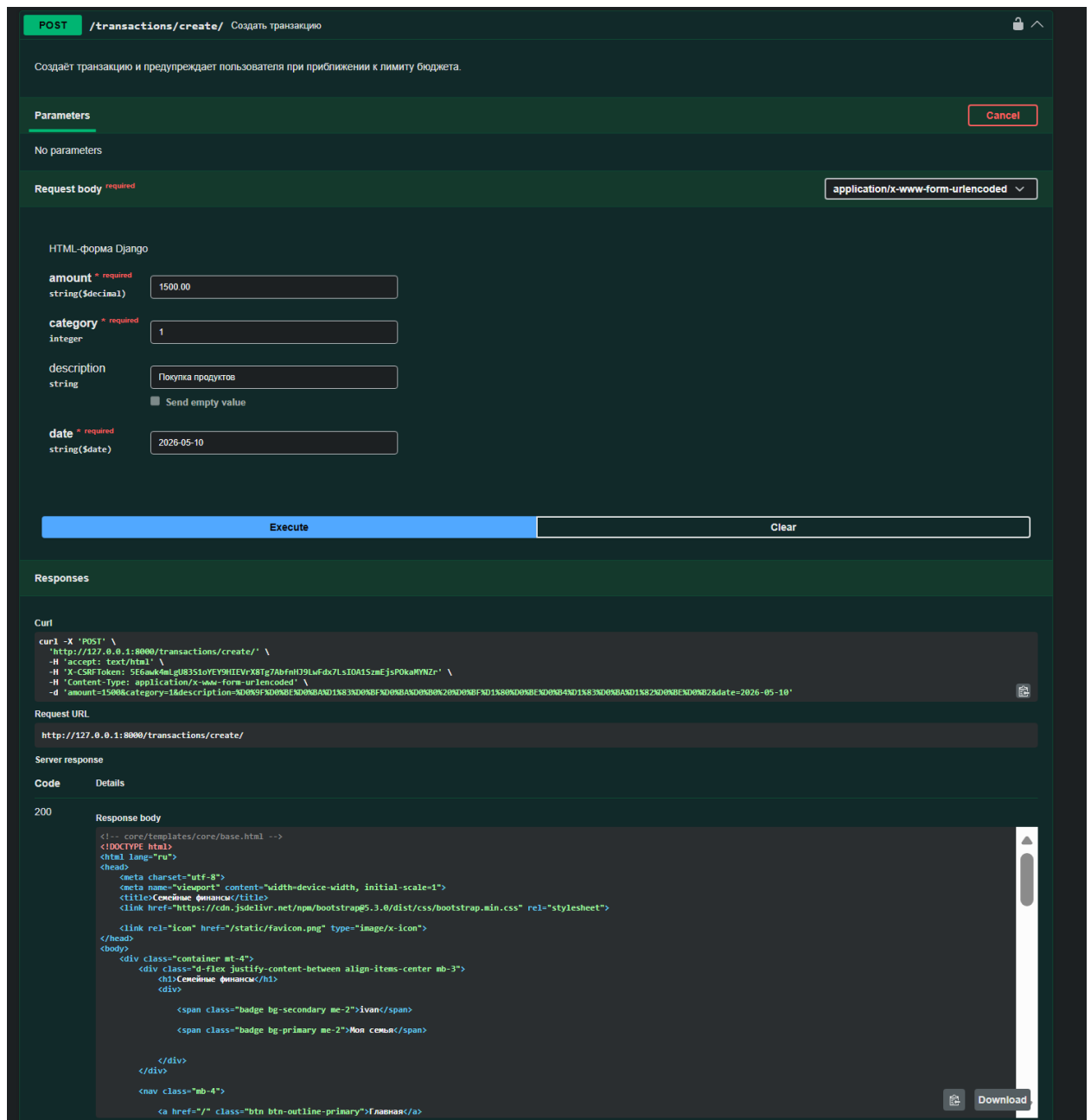


Рисунок 13 – Создание транзакции через Swagger UI



Рисунок 14 – Проверка маршрутов веб-приложения через Swagger UI

7.5 Тестирование API с использованием Postman

Для дополнительной проверки работы серверной части веб-приложения было выполнено тестирование HTTP-запросов с использованием программы Postman. Данный инструмент позволяет вручную отправлять запросы к

серверу, указывать метод запроса, заголовки, параметры и тело запроса, а также анализировать полученный ответ, код состояния и возвращаемые данные.

В рамках тестирования были проверены два основных сценария работы с серверной частью приложения: получение списка финансовых транзакций и создание новой категории. Для проверки получения данных использовался GET-запрос к маршруту `/api/transactions/`, который возвращает список транзакций пользователя в формате JSON. Для проверки создания данных использовался POST-запрос к маршруту `/api/categories/`, позволяющий создать новую категорию доходов или расходов.

Перед выполнением запросов была выполнена авторизация пользователя, так как финансовые данные относятся к личной информации и должны быть доступны только владельцу учетной записи. Это позволяет проверить не только обработку HTTP-запросов, но и корректность работы механизма ограничения доступа к данным.

Первым был выполнен GET-запрос для получения списка транзакций. В результате сервер вернул ответ со статусом 200 OK, что подтверждает успешную обработку запроса. В теле ответа был получен список транзакций в формате JSON, содержащий идентификатор операции, пользователя, категорию, тип категории, сумму, описание и дату операции. Результат выполнения GET-запроса представлен на рисунке 15.

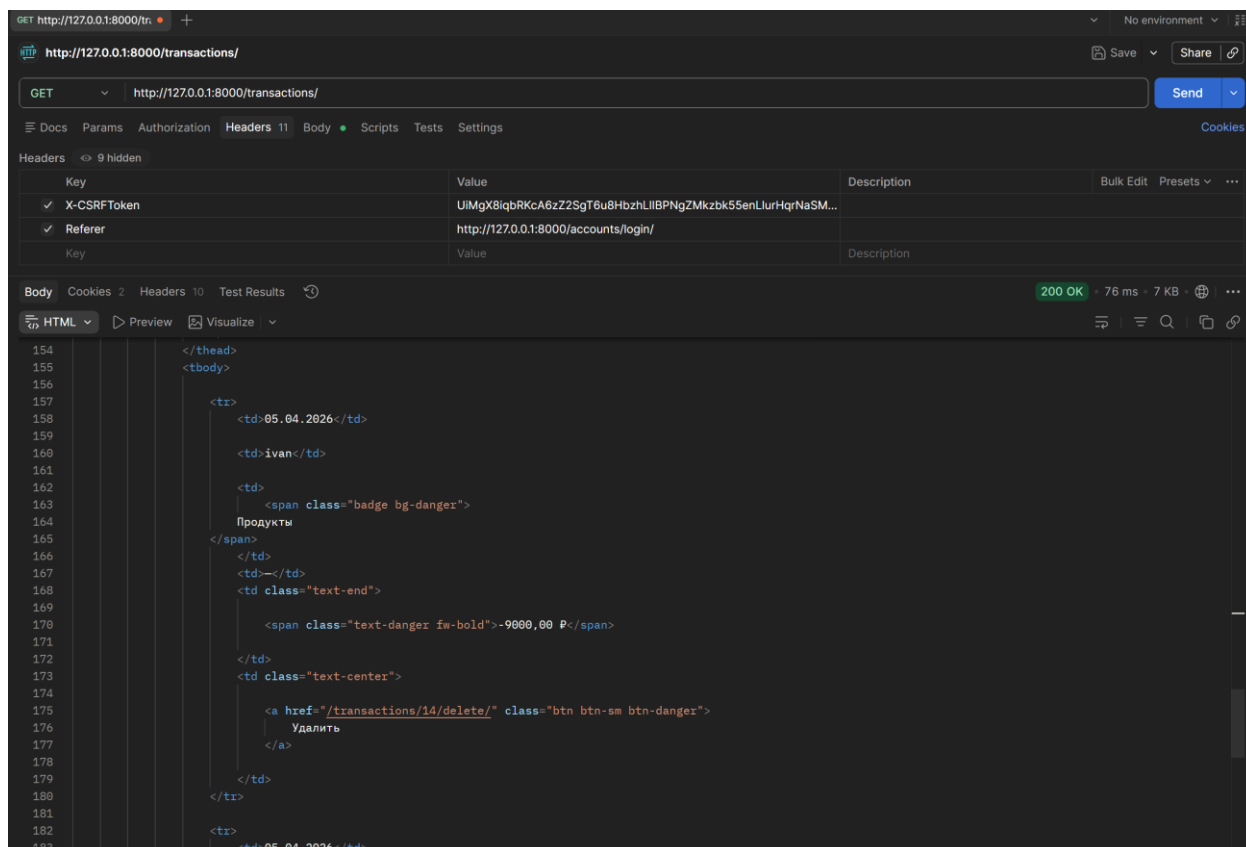


Рисунок 15 – Выполнение GET-запроса для получения списка транзакций в Postman

Далее был выполнен POST-запрос для создания новой категории. В теле запроса были переданы название категории и ее тип. В качестве формата данных использовался JSON. После обработки запроса сервер вернул ответ со статусом 201 Created, что означает успешное создание нового объекта. В теле ответа были отображены идентификатор созданной категории, ее название, тип и сообщение об успешном выполнении операции. Результат выполнения POST-запроса представлен на рисунке 16.

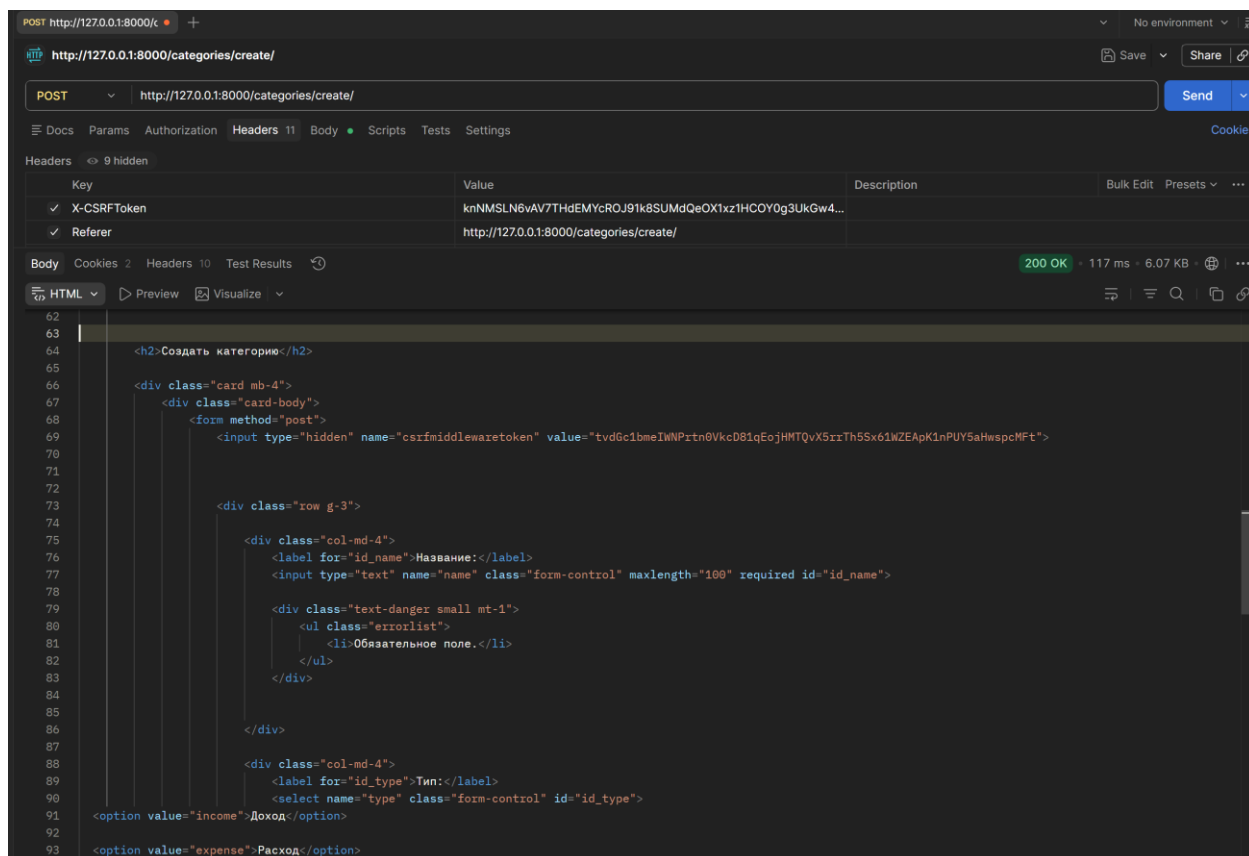


Рисунок 16 – Выполнение POST-запроса для создания категории в Postman

Проведенное тестирование показало, что серверная часть веб-приложения корректно обрабатывает запросы на получение и создание данных. GET-запрос подтвердил возможность получения списка финансовых транзакций в структурированном виде, а POST-запрос подтвердил возможность создания новой категории через серверный API. Таким образом, работа основных API-маршрутов была проверена с помощью Postman.

7.6 Результаты тестирования

По результатам тестирования подтверждена корректная работа основных функций веб-приложения: регистрация и авторизация пользователей, защита страниц от неавторизованного доступа, создание семейной группы, разграничение прав владельца и участника, управление категориями, добавление финансовых транзакций, установка бюджетных ограничений, расчет месячной сводки, формирование отчетности, импорт и экспорт CSV-файлов.

Автоматизированные тесты подтвердили корректность работы моделей,

представлений, сервисных функций и механизмов разграничения прав доступа. Проверка бизнес-логики показала, что приложение корректно рассчитывает доходы, расходы и баланс за выбранный период, определяет превышение бюджета и обрабатывает финансовые операции в соответствии с заданными правилами.

Проверка Swagger UI показала, что OpenAPI-документация формируется корректно и отражает основные маршруты приложения. Анализ покрытия кода с помощью coverage позволил оценить полноту автоматизированного тестирования и выявить участки, которые могут быть дополнительно проверены в дальнейшем.

Таким образом, проведенное тестирование подтверждает соответствие разработанного веб-приложения заявленным функциональным и нефункциональным требованиям. Система корректно обрабатывает основные пользовательские сценарии, обеспечивает разграничение доступа к финансовым данным, поддерживает работу с семейным бюджетом и может быть использована для демонстрации и дальнейшего развития.

Заключение

В ходе выполнения курсовой работы была разработана серверная часть веб-приложения для планирования семейного бюджета. Разрабатываемая система предназначена для учета доходов и расходов, управления семейными группами, контроля бюджетных ограничений и формирования финансовой отчетности.

В рамках работы был проведен анализ предметной области и существующих решений для учета личных и семейных финансов. Были определены основные пользователи системы, функциональные и нефункциональные требования, а также ключевые сущности предметной области.

Для реализации проекта был выбран язык программирования Python и фреймворк Django. В качестве базы данных использовалась SQLite, взаимодействие с которой выполняется через Django ORM. Клиентский слой реализован с помощью Django Templates, а для программного взаимодействия с сервером предусмотрен REST API с документацией Swagger/OpenAPI.

В процессе разработки была спроектирована архитектура веб-приложения на основе паттерна MVT. Реализованы основные серверные модули: регистрация и авторизация пользователей, управление семейными группами, категориями финансовых операций, транзакциями, бюджетами, отчетностью, комментариями и историей изменений.

Также была разработана структура базы данных, включающая пользователей, семейные группы, участников семей, категории, транзакции, бюджеты и дополнительные сущности сопровождения данных. Связи между таблицами обеспечивают целостность информации и позволяют корректно обрабатывать финансовые операции пользователей.

Разработанное веб-приложение обеспечивает выполнение поставленных задач и может использоваться как основа для дальнейшего развития системы.

Список используемой литературы

1. Баланов, А. Н. Бэкенд-разработка веб-приложений: архитектура, проектирование и управление проектами : учебное пособие для вузов / А. Н. Баланов. — 2-е изд., стер. — Санкт-Петербург : Лань, 2025. — 312 с. — ISBN 978-5-507-52472-3. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/451820> (дата обращения: 01.12.2025). — Режим доступа: для авториз. пользователей.
2. Django documentation. — Текст : электронный // Django Software Foundation. — URL: <https://docs.djangoproject.com/> (дата обращения: 10.05.2026).
3. Python 3 documentation. — Текст : электронный // Python Software Foundation. — URL: <https://docs.python.org/3/> (дата обращения: 10.05.2026).
4. SQLite documentation. — Текст : электронный // SQLite. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 10.05.2026).
5. Django REST framework documentation. — Текст : электронный // Encode OSS Ltd. — URL: <https://www.django-rest-framework.org/> (дата обращения: 10.05.2026).
6. drf-spectacular documentation. — Текст : электронный // drf-spectacular. — URL: <https://drf-spectacular.readthedocs.io/> (дата обращения: 10.05.2026).
7. OpenAPI Specification. — Текст : электронный // OpenAPI Initiative. — URL: <https://spec.openapis.org/oas/latest.html> (дата обращения: 10.05.2026).
8. Coverage.py documentation. — Текст : электронный // Ned Batchelder. — URL: <https://coverage.readthedocs.io/> (дата обращения: 10.05.2026).
9. Relax1205. Серверная часть веб-приложения для планирования семейного бюджета: репозиторий проекта. — URL: https://github.com/Relax1205/kursach_6_backend